ACTIVITY-BASED-LEARNING

CASE STUDIES:

1. Progressive Electricity Billing.....05
2. Password Policy Validation Engine.....10
3. Food Waste Monitoring System.....15
4. Student Marks Matrix Analysis 20
5. Smart Parking Fee Management.....25
6. Student Timetable Conflict Detection.....30
7. Smart Canteen Order and Stock Management.....37
8. Sports Tournament Ranking System 44
9. University Scholarship Allocation.....50
10. University Hostel Room Allocation.....57

Progressive Electricity Billing

1.Problem Statement: Write a Python program to calculate an electricity bill based on the number of units consumed using progressive billing slabs

2.Algorithm:

1. Start the program.
2. Read the consumer's name.
3. Read the number of electricity units consumed.
4. Check whether the units are negative.
5. If units are negative, display "Invalid units consumed!"
6. Otherwise, calculate the bill using progressive slabs:
   First 100 units: ₹2 per unit.
   Next 100 units: ₹3 per unit.
   Next 100 units: ₹5 per unit.
   Above 300 units: ₹7 per unit.
7. Display the consumer's name and units consumed.
8. Display the total electricity bill.
9. Stop the program

3.Code:

```
name = input ("Enter consumer name: ")
units = float (input ("Enter units consumed: "))
if units < 0:
    print ("Invalid units consumed!")
else:
    if units <= 100:
        bill = units * 2
    elif units <= 200:
        bill = (100 * 2) + (units - 100) * 3
    elif units <= 300:
        bill = (100 * 2) + (100 * 3) + (units - 200) * 5
    else:
        bill = (100 * 2) + (100 * 3) + (100 * 5) + (units - 300) * 7
    print ("Consumer Name:", name)
    print ("Units Consumed:", units)
    print ("Total Bill: Rs.", round (bill, 2))
```

4.Expected output:

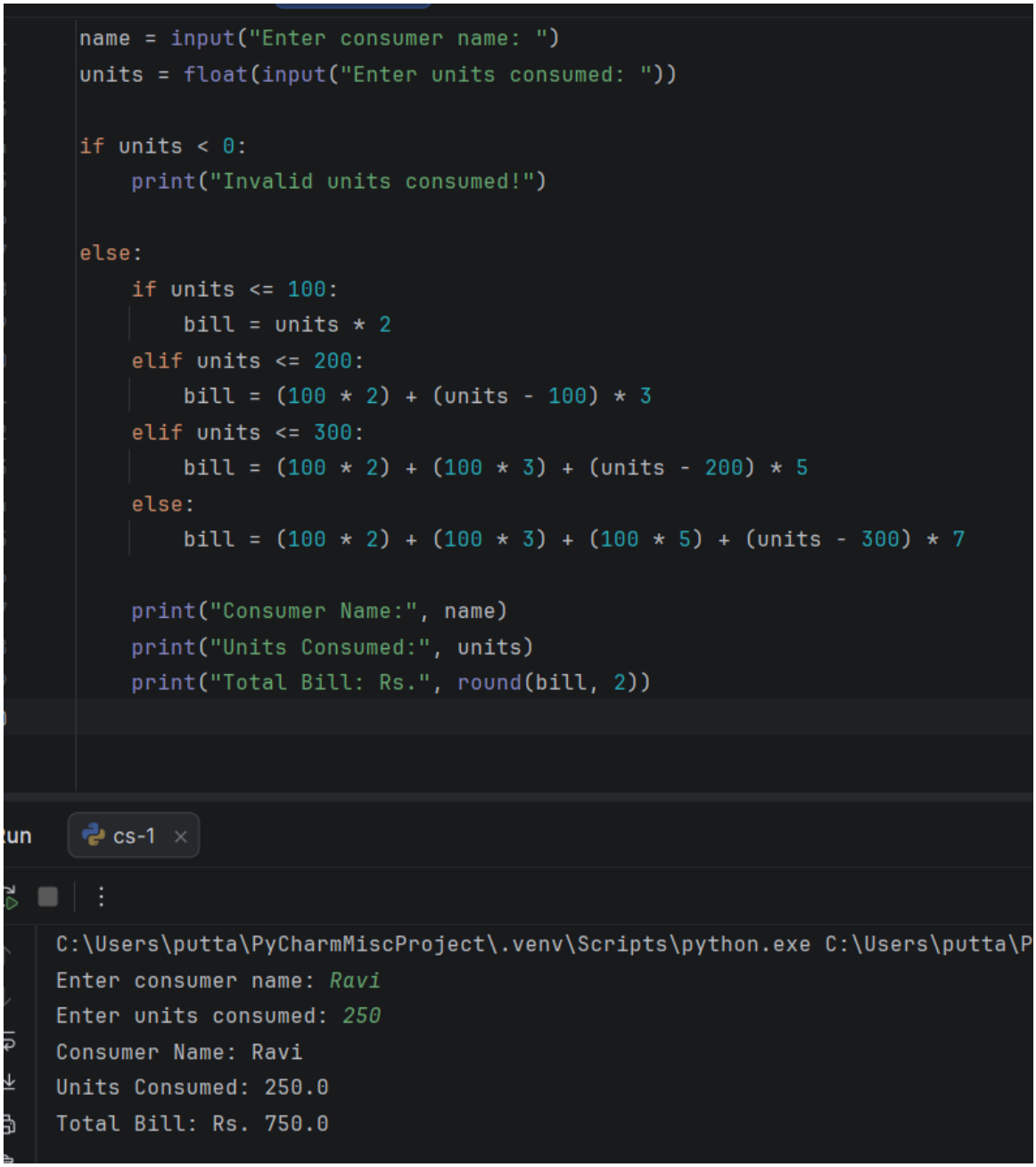
```
Enter consumer name: Ravi
Enter units consumed: 250
Consumer Name: Ravi
Units Consumed: 250.0
Total Bill: Rs. 750.0
```

5.Actual output:

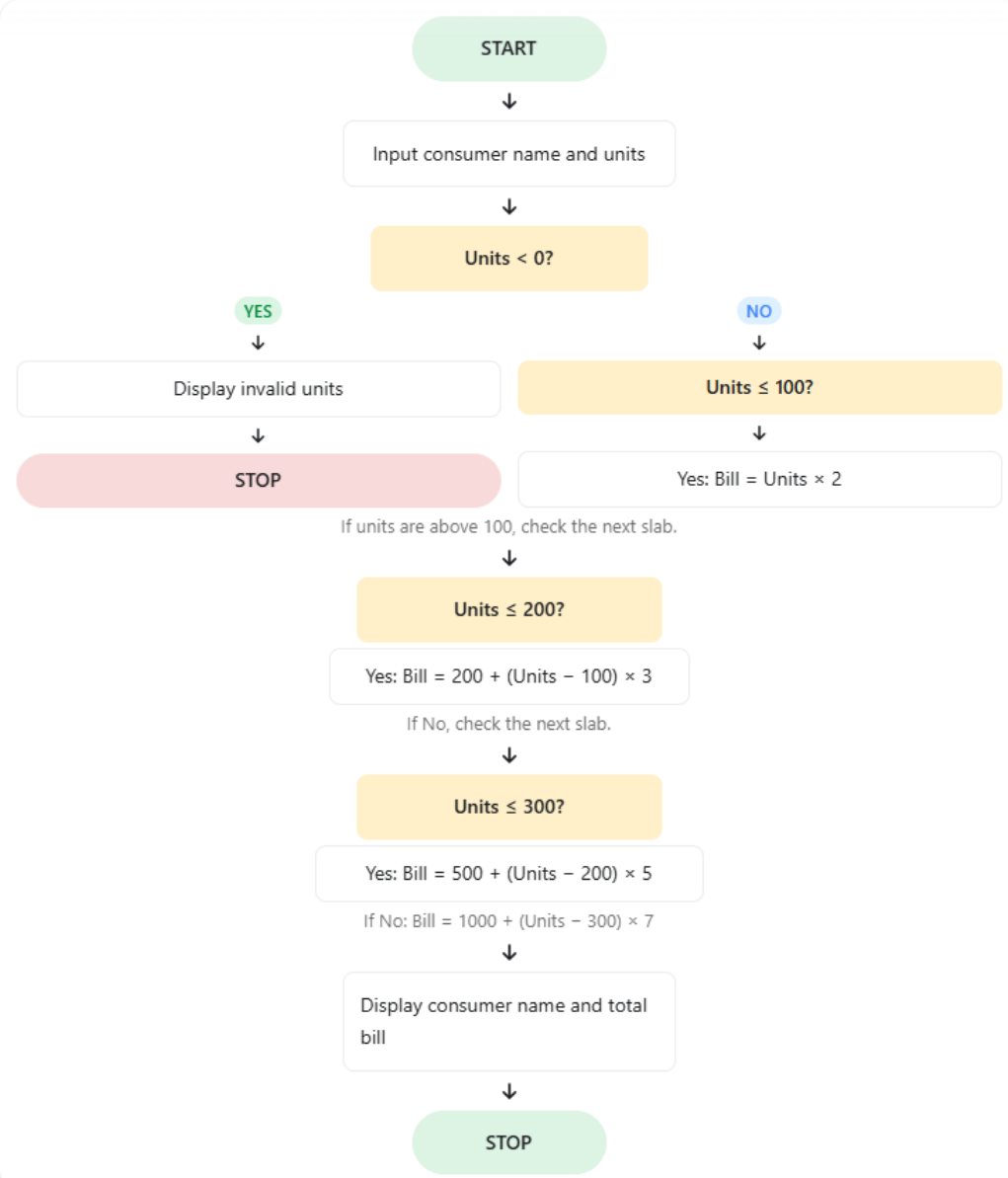
```
Enter consumer name: Ravi
```

```
Enter units consumed: 250
Consumer Name: Ravi
Units Consumed: 250.0
Total Bill: Rs. 750.0
```

6.Code screenshot:



7.Flow chart:



8.Objectives:

1. Calculate an electricity bill using progressive slab rates.

2.Practice Python input, output, arithmetic operators, and if-elif-else statements.

3. Validate the units entered by the user.

4.Display a clear electricity bill report

9.Assumptions:

Units consumed	Rate
First 100 units	₹2 per unit
Next 100 units (101–200)	₹3 per unit
Next 100 units (201–300)	₹5 per unit
Above 300 units	₹7 per unit

10.Pseudocode:

```
BEGIN
 INPUT name
 INPUT units

 IF units < 0 THEN
    DISPLAY "Invalid units consumed!"
 ELSE
   IF units <= 100 THEN
     bill = units * 2
   ELSE IF units <= 200 THEN
     bill = 100 * 2 + (units - 100) * 3
   ELSE IF units <= 300 THEN
     bill = 100 * 2 + 100 * 3 + (units - 200) * 5
   ELSE
      bill = 100 * 2 + 100 * 3 + 100 * 5 + (units - 300) * 7
   END IF

   DISPLAY name, units, bill
 END IF
END
```

11.Test Cases:

Test	Units	Expected bill/result	Purpose
1	50	₹100	Within first slab
2	150	₹350	Within second slab
3	250	₹750	Within third slab
4	350	₹1,350	Above 300 units
5	-5	Invalid units are consumed!	Negative input

12.Conclusion:

The Progressive Electricity Billing program calculates charges based on the number of units consumed and the applicable slab rates. It demonstrates input/output, arithmetic calculations, conditional statements, and basic input validation in Python.

PASSWORD POLICY VALIDATION ENGINE

1. Problem Statement

Passwords help protect computer systems and user accounts. Develop a Python program called Password Policy Validation Engine that accepts a password and checks whether it follows a specified policy. The program checks password length, allowed characters, and whether it contains at least one digit. It displays whether the password is valid and explains any rule that is not satisfied.

2. Objectives

- Validate a password against defined rules.
- Check password length and allow characters.
- Ensure at least one digit is present.
- Practice Python strings, loops, conditions, and Boolean expressions.
- Display clear validation results and helpful error messages.

3. Assumptions:

- A password is valid only if all these rules are satisfied:
- The password must contain 6 to 8 characters.
- Only uppercase English letters (A–Z) and digits (0–9) are allowed.
- The password must contain at least one digit.
- Spaces, lowercase letters, and special characters are not allowed.

4. Algorithm:

1. Start.
2. Input the password.
3. Create an empty list to store error messages.
4. Check whether the password length is between 6 and 8 characters. If not, record a length error.
5. Check that every character is an uppercase letter or a digit. If not, record a character error.
6. Check whether the password contains at least one digit. If not, record a digit requirement error.
7. If no errors were recorded, display “Password is valid.”
8. Otherwise, display “Password is invalid” and show the reasons.
9. Stop.

5. Pseudocode:

```
BEGIN
 INPUT password
 errors = empty list

 IF length < 6 OR length > 8:
 ADD length error to errors
```

```
IF any character is not A-Z or 0-9:
ADD invalid character error to errors

IF password contains no digit:
ADD digit requirement error to errors

IF errors is empty:
DISPLAY "Password is valid"
ELSE:
DISPLAY "Password is invalid"
DISPLAY each error
END
```

6. Python Source Code

```
password = input ("Enter password: ")
errors = []

if Len(password) < 6 or Len(password) > 8:
 errors. Append ("Password must be 6 to 8 characters long.")

if not all (('A' <= ch <= 'Z') or ('0' <= ch <= '9')
 for ch in password):
 errors. Append ("Use uppercase letters (A-Z) and digits (0-9).")

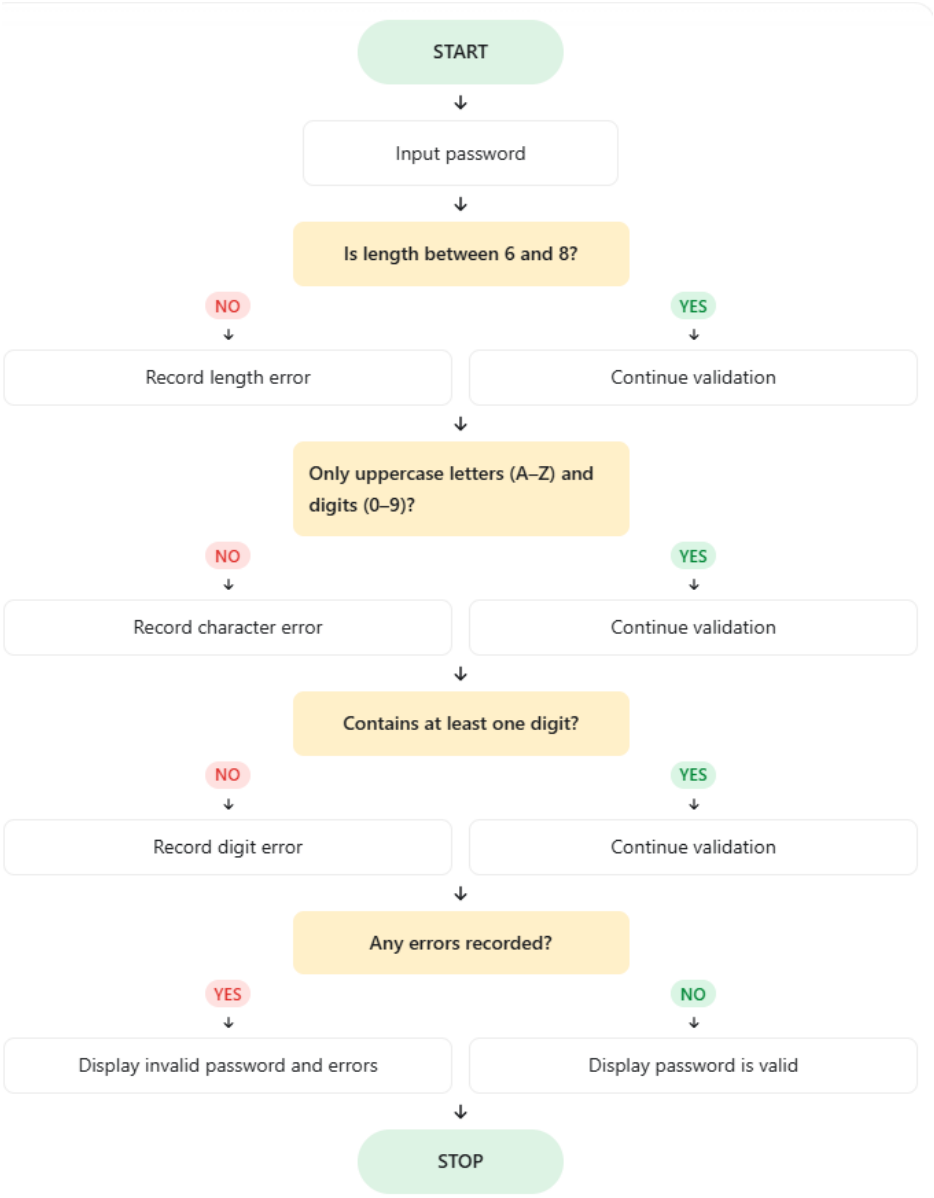
if not any ('0' <= ch <= '9' for ch in password):
 errors. Append ("Password must contain at least one digit.")

if Len(errors) == 0:
  print ("Password is valid.")
else:
 print ("Password is invalid.")
 for error in errors:
 print ("-", error)
```

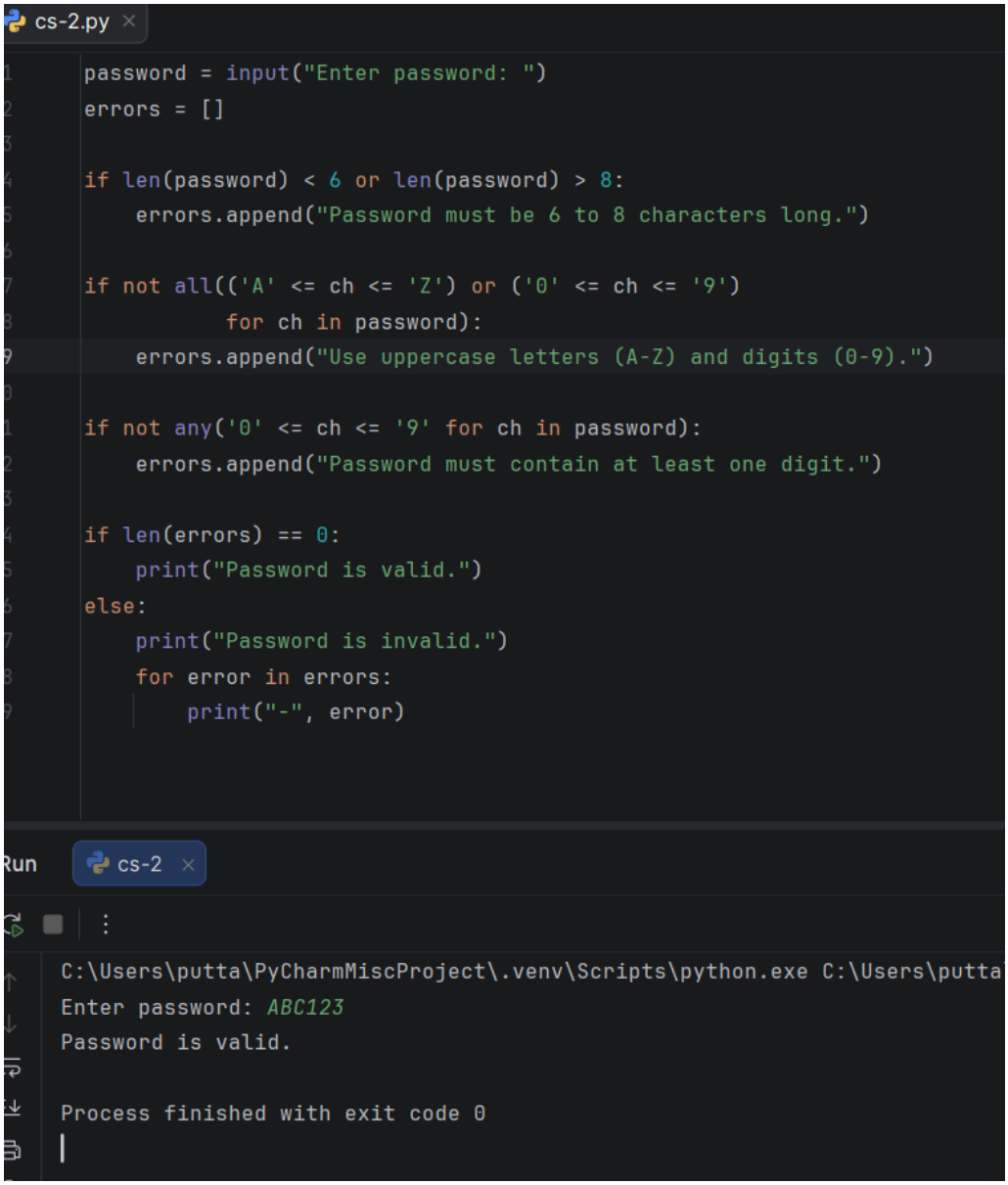
7. Test Cases:

Test	Input	Expected result	Reason
1	ABC123	Valid	6 characters; uppercase and digits; has a digit
2	AB12	Invalid	Fewer than 6 characters
3	ABCDEFG	Invalid	No digit
4	AbC123	Invalid	Lowercase character
5	ABC@123	Invalid	Special character
6	ABCDEFG12	Invalid	More than 8 characters

8.flowchart:



9.output screenshot:



10.expected output:

Enter password: ABC123

Passwords are valid.

11.actual output:

Enter password: ABC123

Passwords are valid.

12. Conclusion

The Password Policy Validation Engine checks whether a password satisfies the specified length, character, and digit requirements. It demonstrates Python string operations, loops, conditional statements, and lists, and gives clear feedback when a password does not meet the policy.

Food Waste Monitoring System

1. Problem Statement

Food waste is a common problem in restaurants, college canteens, hostels, and homes. Excess food preparation wastes ingredients, money, and resources. This Python program records food prepared and food wasted, calculates food consumed and waste percentage, and suggests ways to reduce waste. It validates quantities before calculating the report.

2. Objectives

• Record food prepared and wasted in kilograms.
• Calculate the amount of food consumed.
• Calculate the percentage of food wasted.
• Validate quantities to avoid incorrect calculations.
• Display a waste status and helpful suggestions.

3. Assumptions

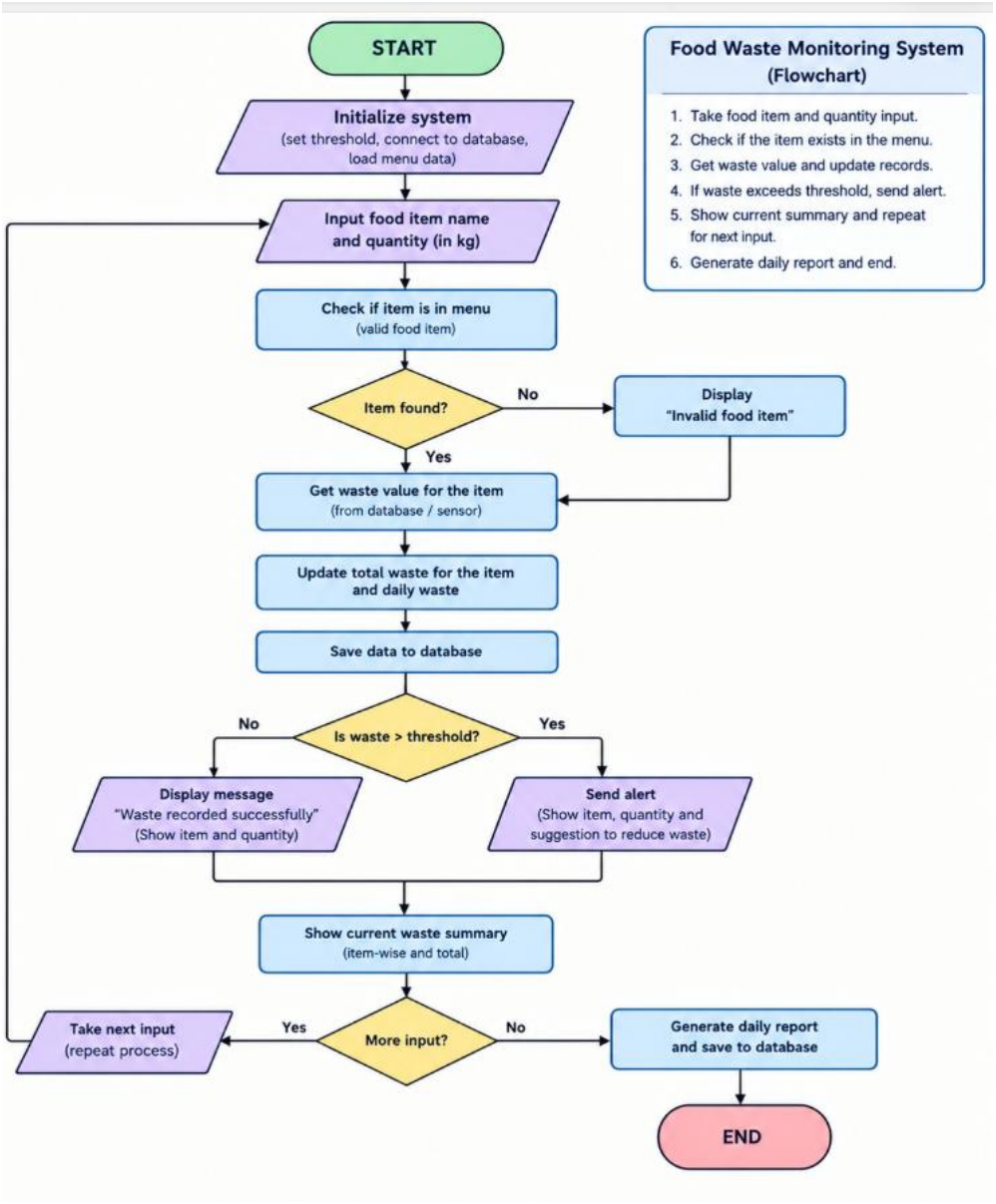
• Food prepared must be greater than zero.
• Food wasted cannot be negative or greater than food prepared.
• Food consumed = Food prepared − Food wasted.
• Waste percentage = (Food wasted ÷ Food prepared) × 100.
• At most 10% = Low; above 10% and at most 25% = Moderate; above 25% = High.

Waste percentage Status Suggestion
0%–10% Low Food waste is low. Continue monitoring.
>10%–25% Moderate Prepare food according to demand.
>25% High Reduce excess preparation and review demand.

4. Algorithm

1. Start.
2. Input food prepared in kg.
3. Input food wasted in kg.
4. If food prepared is less than or equal to zero, display an invalid prepared quantity message.
5. Else, if food wasted is negative or greater than food prepared, display an invalid waste quantity message.
6. Otherwise calculate food consumed = prepared − wasted.
7. Calculate waste percentage = (wasted ÷ prepared) × 100.
8. Display prepared, consumed, wasted, and waste percentage.
9. If waste percentage ≤ 10, display Low; else if ≤ 25, display Moderate and advice; otherwise display High and advice.
10. Stop.

5. Flowchart



6. Pseudocode

BEGIN
INPUT food_prepared
INPUT food_wasted
IF food_prepared <= 0:
DISPLAY invalid prepared quantity
ELSE IF food_wasted < 0 OR food_wasted > food_prepared:
DISPLAY invalid wasted quantity
ELSE:
food_consumed = food_prepared - food_wasted
waste_percentage = (food_wasted / food_prepared) * 100
DISPLAY food report
IF waste_percentage <= 10:
DISPLAY LOW waste
ELSE IF waste_percentage <= 25:

DISPLAY MODERATE waste and suggestion
ELSE:
DISPLAY HIGH waste and suggestion
END IF
END

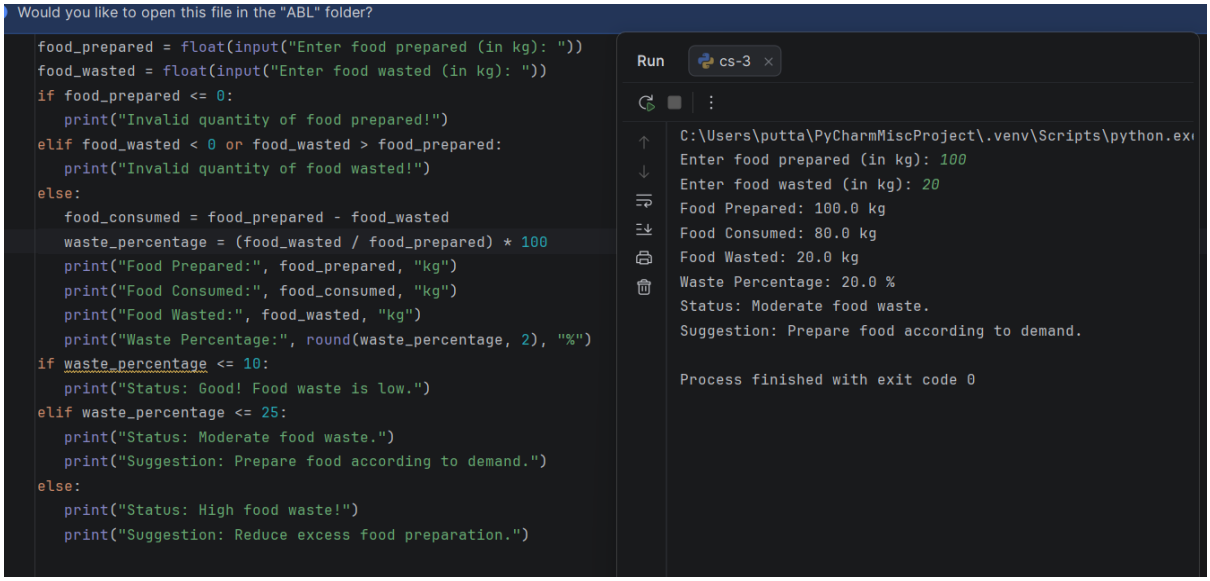
7. Python Source Code

```
food_prepared = float(input("Enter food prepared (in kg): "))
food_wasted = float(input("Enter food wasted (in kg): "))
if food_prepared <= 0:
    print("Invalid quantity of food prepared!")
elif food_wasted < 0 or food_wasted > food_prepared:
    print("Invalid quantity of food wasted!")
else:
    food_consumed = food_prepared - food_wasted
    waste_percentage = (food_wasted / food_prepared) * 100
    print("Food Prepared:", food_prepared, "kg")
    print("Food Consumed:", food_consumed, "kg")
    print("Food Wasted:", food_wasted, "kg")
    print("Waste Percentage:", round(waste_percentage, 2), "%")
    if waste_percentage <= 10:
       print("Status: Good! Food waste is low.")
    elif waste_percentage <= 25:
        print ("Status: Moderate food waste.")
        print ("Suggestion: Prepare food according to demand.")
   else:
print("Status: High food waste!")
print("Suggestion: Reduce excess food preparation.")
```

8. Test Cases

test	prepared	wasted	Expected result
1	100kg	5kg	Consumed 95 kg; waste 5%; Low
2	100kg	20kg	Consumed 80 kg; waste 20%; Moderate
3	100kg	40kg	Consumed 60 kg; waste 40%; Hig
4	0kg	0kg	Invalid quantity of food prepared
5	50kg	-2kg	Invalid quantity of food wasted

9.output screenshot:



10.expected output:

Enter food prepared (in kg): 100

Enter food wasted (in kg): 20

Food Prepared: 100.0 kg

Food Consumed: 80.0 kg

Food Wasted: 20.0 kg

Waste Percentage: 20.0 %

Status: Moderate food waste.

Suggestion: Prepare food according to demand.

11.Actual output:

Enter food prepared (in kg): 100

Enter food wasted (in kg): 20

Food Prepared: 100.0 kg

Food Consumed: 80.0 kg

Food Wasted: 20.0 kg

Waste Percentage: 20.0 %

Status: Moderate food waste.

Suggestion: Prepare food according to demand.

12. Conclusion

The Food Waste Monitoring System calculates the quantity and percentage of wasted food, classifies waste as low, moderate, or high, and offers simple suggestions. It demonstrates Python input/output, arithmetic calculations, conditional statements, and input validation. It can help canteens and restaurants make better food preparation decisio_n

STUDENT MARKS MATRIX ANALYSIS

1. Problem Statement

Schools and colleges need to organize marks for several students across multiple subjects. Calculating totals, averages, subject performance, and the highest-scoring student manually take time and can cause errors. This Python program stores marks in a two-dimensional list (matrix), validates marks, calculates each student's total and average, calculates subject-wise averages, and identifies the student with the highest total.

2. Objectives

- Store marks in a two-dimensional list.
- Validate that each mark is between 0 and 100.
- Calculate each student's total and average.
- Calculate each subject's average.
- Find the student with the highest total marks.

3. Assumptions

- The example uses 3 students and 3 subjects; the program can accept other positive counts.
- Marks must be whole numbers from 0 to 100 inclusive.
- Each matrix row represents a student; each column represents a subject.
- Student total = sum of student marks.
- Student average = total ÷ number of subjects.
- Subject average = sum of that subject's marks ÷ number of students.
- If totals are tied, the first student with the highest total is reported.

4. Algorithm

1. Start.
2. Input the number of students and subjects.
3. Input student names and subject names.
4. Input marks for each student and subject.
5. If a mark is outside 0–100, ask for it again.
6. Store valid marks on a two-dimensional list.
7. Calculate each student's total and average.
8. Calculate the average for each subject.
9. Find the student with the highest total.
10. Display the matrix, totals, averages, subject averages, and highest-scoring student.
11. Stop.

5.flowchart:

6. Pseudocode

```
BEGIN
INPUT number_of_  students, number_of_subjects
INPUT student names and subject names
marks = empty 2D list
FOR each student:
row = empty list
FOR each subject:
```

```
INPUT mark
WHILE mark < 0 OR mark > 100:
DISPLAY "Enter a mark from 0 to 100"
INPUT mark again
ADD mark to row
ADD row to marks
FOR each student:
total = sum of marks in the row
average = total / number_of_subjects
DISPLAY total and average
FOR each subject:
calculate and display subject average
FIND student with highest total
DISPLAY analysis report
END
```

7. Python Source Code

```
n = int(input("Enter number of students: "))
m = int(input("Enter number of subjects: "))
if n <= 0 or m <= 0:
print("Students and subjects must be positive numbers.")
else:
students = [input(f" Student {i+1} name: ") for i in range(n)]
subjects = [input(f" Subject {j+1} name: ") for j in range(m)]
marks = []
for i in range(n):
row = []
print("Enter marks for", students[i])
for j in range(m):
mark = int(input(subjects[j] + ": "))
while mark < 0 or mark > 100:
print("Invalid mark. Enter 0 to 100.")
mark = int(input(subjects[j] + ": "))
row.append(mark)
marks.append(row)
totals = [sum(row) for row in marks]
print("\n STUDENT RESULTS")
for i in range(n):
print(students[i], "Marks:", marks[i],
```

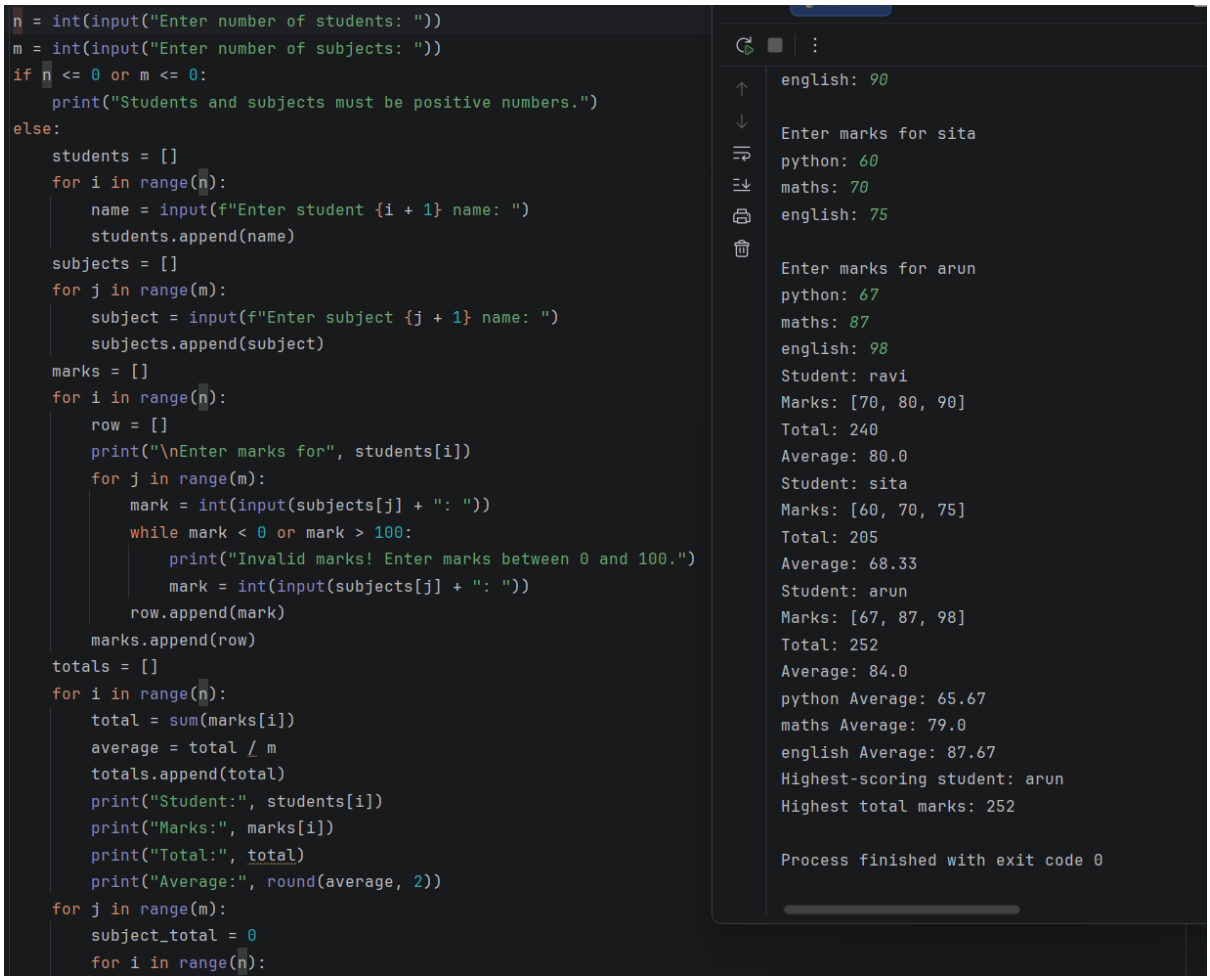


```
"Total:", totals[i],
"Average:", round(totals[i] / m, 2))
print("\n SUBJECT AVERAGES")
for j in range(m):
avg = sum(marks[i][j] for i in range(n)) / n
print(subjects[j], ":", round (avg, 2))
top = totals. index(max(totals))
print("\n Highest-scoring student:", students[top])
print("Highest total:", totals[top])
```

8. Test Cases

Test Input / situation Expected result
1 Marks 80, 90, 70 Total 240; average 80.00
2 Marks 100, 100, 100 Total 300; average 100.00
3 Enter -5 as a mark Reject and ask again
4 Enter 105 as a mark Reject and ask again
5 Two students tie for top total First student with that total is displayed
6 Number of students = 0 Validation message displayed

9.output screenshot:



```
n = int(input("Enter number of students: "))
m = int(input("Enter number of subjects: "))
if n <= 0 or m <= 0:
    print("Students and subjects must be positive numbers.")
else:
    students = []
    for i in range(n):
        name = input(f"Enter student {i + 1} name: ")
        students.append(name)
    subjects = []
    for j in range(m):
        subject = input(f"Enter subject {j + 1} name: ")
        subjects.append(subject)
    marks = []
    for i in range(n):
        row = []
        print("\nEnter marks for", students[i])
        for j in range(m):
            mark = int(input(subjects[j] + ": "))
            while mark < 0 or mark > 100:
                print("Invalid marks! Enter marks between 0 and 100.")
                mark = int(input(subjects[j] + ": "))
            row.append(mark)
        marks.append(row)
    totals = []
    for i in range(n):
        total = sum(marks[i])
        average = total / m
        totals.append(total)
        print("Student:", students[i])
        print("Marks:", marks[i])
        print("Total:", total)
        print("Average:", round(average, 2))
    for j in range(m):
        subject_total = 0
        for i in range(n):
```

```
english: 90

Enter marks for sita
python: 60
maths: 70
english: 75

Enter marks for arun
python: 67
maths: 87
english: 98
Student: ravi
Marks: [70, 80, 90]
Total: 240
Average: 80.0
Student: sita
Marks: [60, 70, 75]
Total: 205
Average: 68.33
Student: arun
Marks: [67, 87, 98]
Total: 252
Average: 84.0
python Average: 65.67
maths Average: 79.0
english Average: 87.67
Highest-scoring student: arun
Highest total marks: 252

Process finished with exit code 0
```

10.expected output:

Enter number of students: 3
Enter number of subjects: 3
Enter student 1 name: ravi
Enter student 2 name: sita
Enter student 3 name: arun
Enter subject 1 name: python
Enter subject 2 name: maths
Enter subject 3 name: english

Enter marks for ravi
python: 70
maths: 80

english: 90

Enter marks for Sita
python: 60
math: 70
English: 75

Enter marks for Arun
python: 67
math: 87
English: 98
Student: Ravi
Marks: [70, 80, 90]
Total: 240
Average: 80.0
Student: sita
Marks: [60, 70, 75]
Total: 205
Average: 68.33
Student: arun
Marks: [67, 87, 98]
Total: 252
Average: 84.0
python Average: 65.67
maths Average: 79.0
English Average: 87.67
Highest-scoring student: arun
Highest total marks: 252

11.actual output:

Enter number of students: 3
Enter number of subjects: 3
Enter student 1 name: ravi
Enter student 2 name: Sita
Enter student 3 name: arun
Enter subject 1 name: python
Enter subject 2 name: maths
Enter subject 3 name: english

Enter marks for ravi
python: 70
maths: 80
english: 90

Enter marks for Sita
python: 60
maths: 70
english: 75

Enter marks for arun

python: 67
maths: 87
english: 98
Student: ravi
Marks: [70, 80, 90]
Total: 240
Average: 80.0
Student: sita
Marks: [60, 70, 75]
Total: 205
Average: 68.33
Student: arun
Marks: [67, 87, 98]
Total: 252
Average: 84.0
python Average: 65.67
maths Average: 79.0
english Average: 87.67
Highest-scoring student: arun
Highest total marks: 252

12. Conclusion

The Student Marks Matrix Analysis program stores marks in a two-dimensional list and calculates student totals, averages, subject-wise averages, and the highest-scoring student. It validates marks from 0 to 100 and demonstrates Python lists, nested loops, conditions, and arithmetic operations

Smart Parking Fee Management

1. Problem Statement:

Manual parking fee calculation can be slow and inconsistent, especially when many vehicles enter and leave a parking facility. Staff may apply the wrong rate, calculate the duration incorrectly, or make arithmetic errors. The goal is to build a small Python program that calculates parking fees consistently from the vehicle category and number of hours parked, then displays the details in a readable receipt.

2. Objectives:

- Calculate parking charges automatically from the selected vehicle type and parking duration.
- Use different hourly rates for two-wheelers, cars, and heavy vehicles.
- Reject invalid vehicle categories and non-positive parking durations.
- Display the vehicle category, duration, hourly rate, and total amount clearly.
- Demonstrate a program that is easy to test, maintain, and extend.

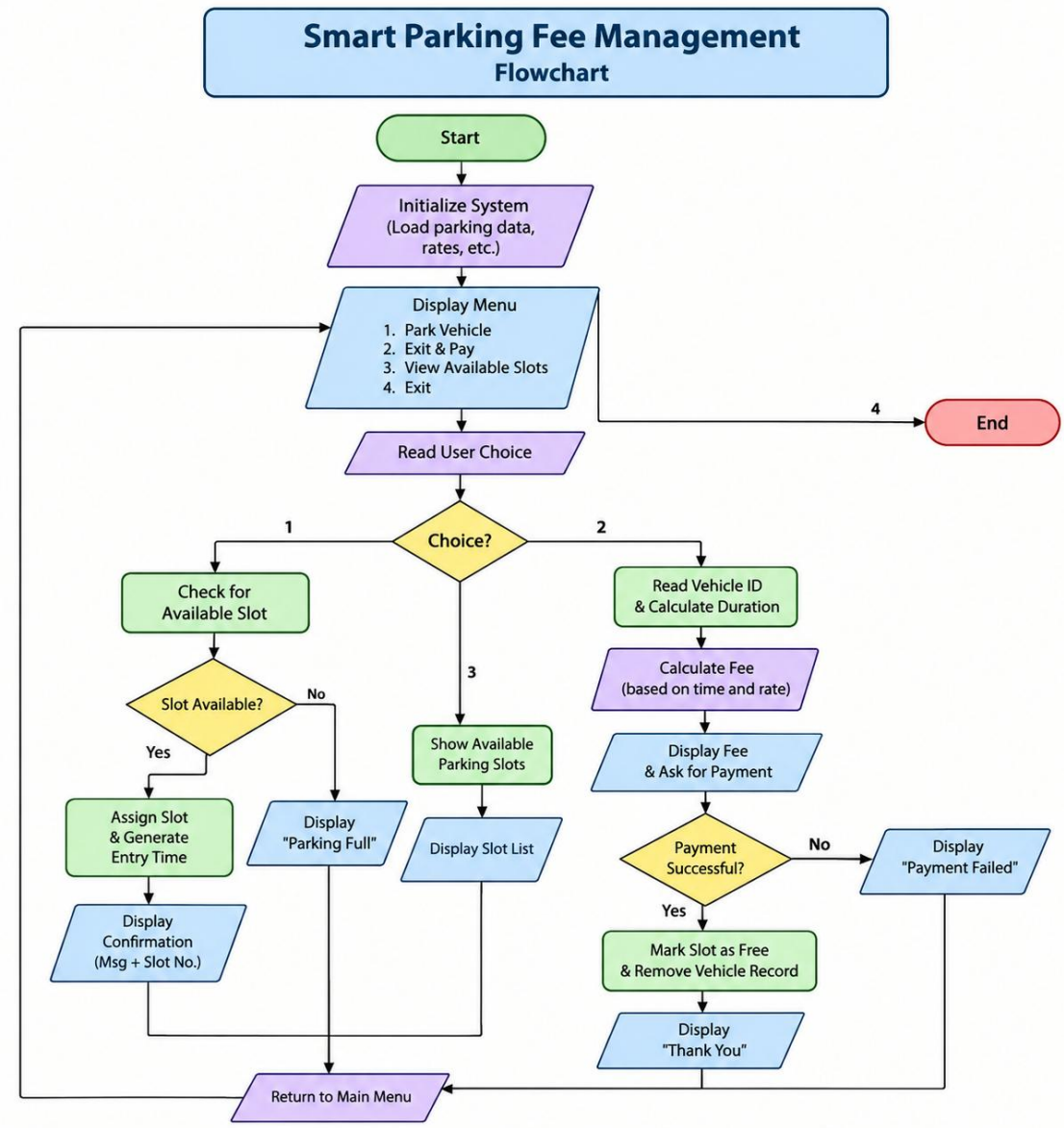
3. Assumptions:

- The user enters one vehicle category and the total parking duration in whole hours.
- The demo supports three categories: Two-wheeler, Car, and Heavy vehicle.
- Rates are sample values in Indian rupees (n), chosen for demonstration only.
- Every started hour is treated as a full hour in this version; the input is therefore an integer number of hours.
- No discounts, taxes, lost-ticket fees, or maximum daily caps are applied.
- The program calculates a fee only; it does not collect payment or store customer information

4.algorithm:

1. Start the program.
2. Display the available vehicle categories and sample hourly rates.
3. Read the vehicle category and normalize the text (remove extra spaces and convert to lowercase).
4. Check whether the category exists in the rate table; if not, show an error and stop.
5. Read the parking duration in hours and check that it is a positive integer.
6. Find the hourly rate for the selected category.
7. Calculate total fee = duration × hourly rate.
8. Print a receipt containing category, duration, rate, and total fee.
9. Stop

5.flowchart:



6. Pseudocode:

BEGIN
rates = {two-wheeler: 10, car: 30, heavy vehicle: 50}
DISPLAY available vehicle categories
INPUT vehicle_type
vehicle_type = lowercase(trim(vehicle_type))
IF vehicle_type is not in rates THEN
DISPLAY "Invalid vehicle category"
STOP
END IF
INPUT parking_hours
Smart Parking Fee Management | Python Activity-Based Learning Page 4
IF parking_hours is not a positive integer THEN
DISPLAY "Enter a positive whole number of hours"
STOP
END IF

```
hourly_rate = rates[vehicle_type]
total_fee = parking_hours * hourly_rate
DISPLAY vehicle_type, parking_hours, hourly_rate, total_fee
END
```

7. Python Source Code:

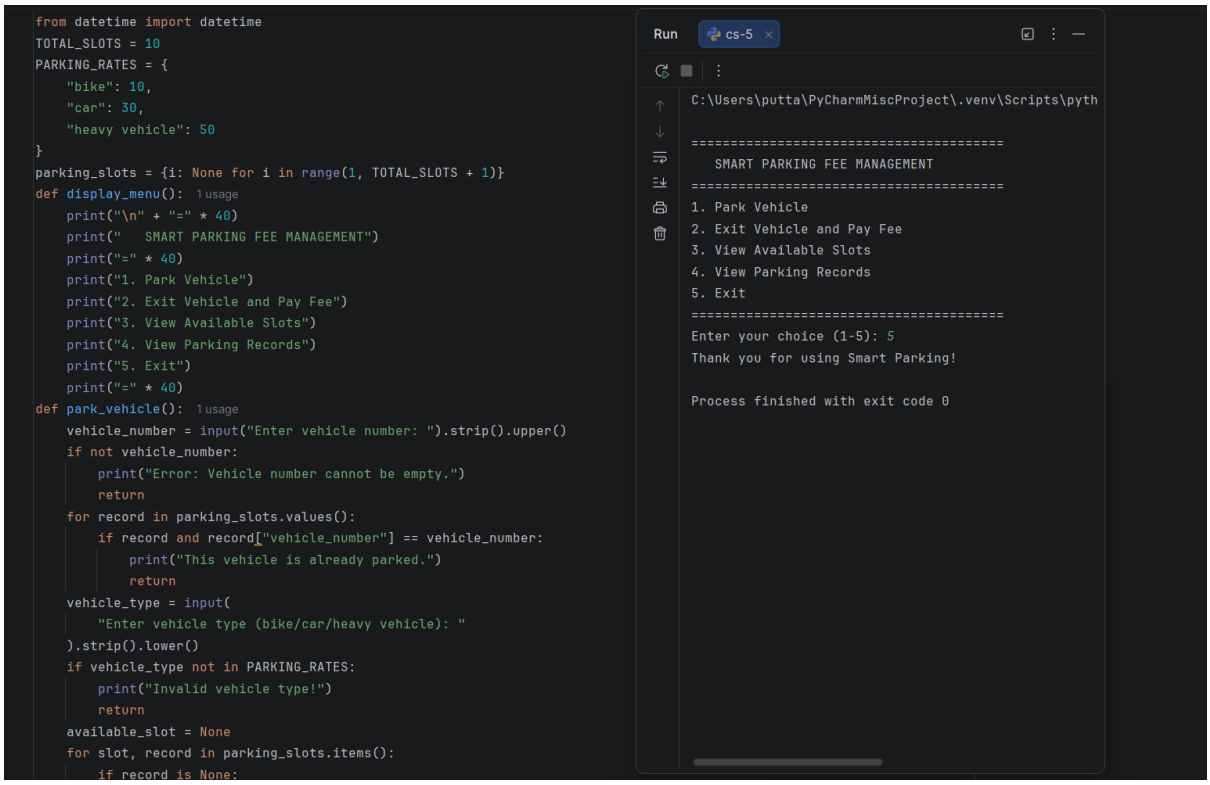
```
RATES = {
"two-wheeler": 10,
"car": 30,
"heavy vehicle": 50
}
def calculate_fee(vehicle_type, parking_hours):
"""Return the total parking fee for a valid category and duration."""
vehicle_type = vehicle_type.strip().lower()
if vehicle_type not in RATES:
raise ValueError("Invalid vehicle category.")
if not isinstance(parking_hours, int) or parking_hours <= 0:
raise ValueError("Parking hours must be a positive whole number.")
return RATES[vehicle_type] * parking_hours
def main():
print("SMART PARKING FEE MANAGEMENT")
print("Vehicle types: two-wheeler, car, heavy vehicle")
vehicle_type = input("Enter vehicle type: ").strip().lower()
if vehicle_type not in RATES:
print("Error: Invalid vehicle category.")
return
hours_text = input("Enter parking duration in whole hours: ").strip()
try:
parking_hours = int(hours_text)
total_fee = calculate_fee(vehicle_type, parking_hours)
except ValueError as error:
print(f"Error: {error}")
return
print(f"Vehicle type : {vehicle_type.title()}")
print(f"Duration : {parking_hours} hour(s)")
print(f"Hourly rate : Rs. {RATES[vehicle_type]}")
print(f"Total fee : Rs. {total_fee}")
print("Thank you. Please proceed to payment.")
if __name__ == "__main__":
main()
```

8. Test Cases:

TC-01 two-wheeler 2 Success; total n20
TC-02 car 3 Success; total n90
TC-03 heavy vehicle 4 Success; total n200
TC-04 car 1 Success; total n30
TC-05 spaceship 2 Error: invalid vehicle category
TC-06 car 0 Error: duration must be positive
TC-07 car -2 Error: duration must be positive

TC-08 car two Error: enter a whole number
TC-09 two-wheeler 1 Success; total n10

9.output scrrenshot:



```
from datetime import datetime
TOTAL_SLOTS = 10
PARKING_RATES = {
    "bike": 10,
    "car": 30,
    "heavy vehicle": 50
}
parking_slots = {i: None for i in range(1, TOTAL_SLOTS + 1)}
def display_menu():  1 usage
    print("\n" + "=" * 40)
    print("   SMART PARKING FEE MANAGEMENT")
    print("=" * 40)
    print("1. Park Vehicle")
    print("2. Exit Vehicle and Pay Fee")
    print("3. View Available Slots")
    print("4. View Parking Records")
    print("5. Exit")
    print("=" * 40)
def park_vehicle():  1 usage
    vehicle_number = input("Enter vehicle number: ").strip().upper()
    if not vehicle_number:
        print("Error: Vehicle number cannot be empty.")
        return
    for record in parking_slots.values():
        if record and record["vehicle_number"] == vehicle_number:
            print("This vehicle is already parked.")
            return
    vehicle_type = input(
        "Enter vehicle type (bike/car/heavy vehicle): "
    ).strip().lower()
    if vehicle_type not in PARKING_RATES:
        print("Invalid vehicle type!")
        return
    available_slot = None
    for slot, record in parking_slots.items():
        if record is None:
```

```
Run   cs-5

C:\Users\putta\PyCharmMiscProject\.venv\Scripts\pyth

========================================
   SMART PARKING FEE MANAGEMENT
========================================
1. Park Vehicle
2. Exit Vehicle and Pay Fee
3. View Available Slots
4. View Parking Records
5. Exit
========================================
Enter your choice (1-5): 5
Thank you for using Smart Parking!

Process finished with exit code 0
```

10.Expected output:

```
========================================
  SMART PARKING FEE MANAGEMENT
========================================
1. Park Vehicle
2. Exit Vehicle and Pay Fee
3. View Available Slots
4. View Parking Records
5. Exit
========================================
Enter your choice (1-5): 5
Thank you for using Smart Parking!
```

11. Actual output:

```
========================================
  SMART PARKING FEE MANAGEMENT
========================================
1. Park Vehicle
2. Exit Vehicle and Pay Fee
3. View Available Slots
4. View Parking Records
5. Exit
========================================
```

Enter your choice (1-5): 5
Thank you for using Smart Parking!

12. Conclusion:

The Smart Parking Fee Management case study demonstrates how Python can automate a common calculation task. The program accepts a vehicle category and parking duration, validates the inputs, applies the relevant sample rate, and displays the total fee. It meets the core learning objectives through functions, dictionaries, conditional logic, exception handling, arithmetic, and formatted output. With additional features such as timestamps, a database, and payment integration, the prototype could be expanded into a more complete parking management system

Student Timetable Conflict Detection

1.Problem statement

Enter each class's course name, day, start hour, and end hour. The program compares each pair and reports a conflict when the days match and the time intervals overlap. Hours are whole numbers from 0 to 23, and each class must end after it starts. Back-to-back classes do not conflict.

2.Assumptions

- Each class occurs on one named weekday and uses whole hours from 0 through 23.
- The end hour must be later than the start hour; classes that cross midnight are outside this version.
- Day names are compared without regard to letter case, and a class ending exactly when another starts is not a conflict.

3.Objectives

1. Detect every pair of classes scheduled on the same day with overlapping hours.
2. Validate class count and time values before checking the timetable.
3. Practice lists, dictionaries, nested loops, and interval comparison.

4.Algorithm

1. Read and validate the number of classes.
2. For each class, read its course, day, start hour, and end hour. Require the end hour to be later than the start hour.
3. Compare each class with every later class so each pair is checked once.
4. A conflict exists when the days match and start1 < end2 and start2 < end1.
5. Store conflicting pairs. Display them or report that none were found.

5.Pseudocode

```
BEGIN
 READ number_of_classes (must be 1 through 30)
 classes = empty list

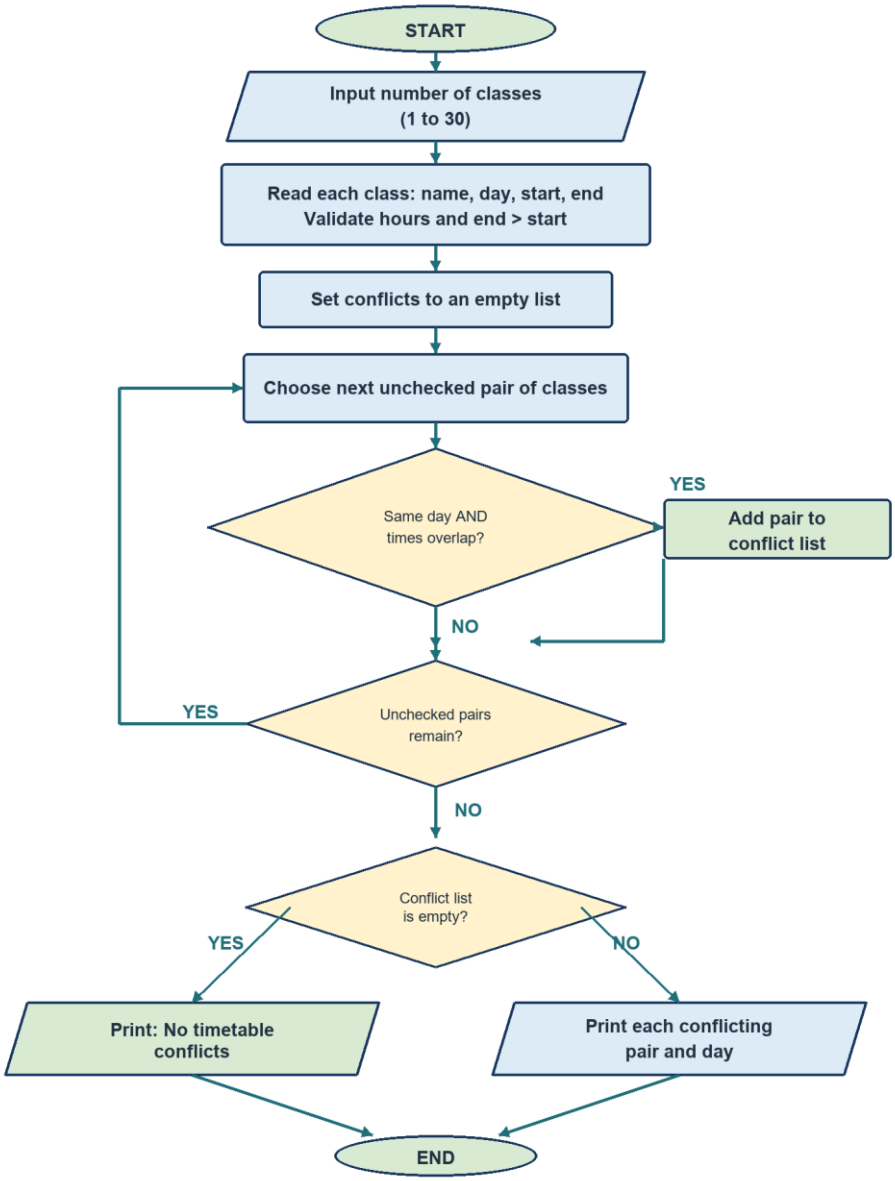

 FOR each class
 READ course_name and day
 CONVERT day to lowercase
 REPEAT
 READ start_hour and end_hour
 IF both hours are from 0 through 23 AND end_hour > start_hour
 STOP repeating
 ELSE
 DISPLAY validation message
```

```
 ADD {name, day, start_hour, end_hour} to classes


 conflicts = empty list
 FOR i from 0 to number of classes - 1
 FOR j from i + 1 to number of classes - 1
 first = classes[i]
 second = classes[j]
 same day = first. Day equals second.day
 overlap = first.start < second. End AND second. Start < first.end
 IF same_day AND overlap
 ADD the pair to conflicts


 IF conflicts is empty
 DISPLAY "No timetable conflicts."
 ELSE
 DISPLAY each conflicting course pair and day
END
```

6.Flowchart:



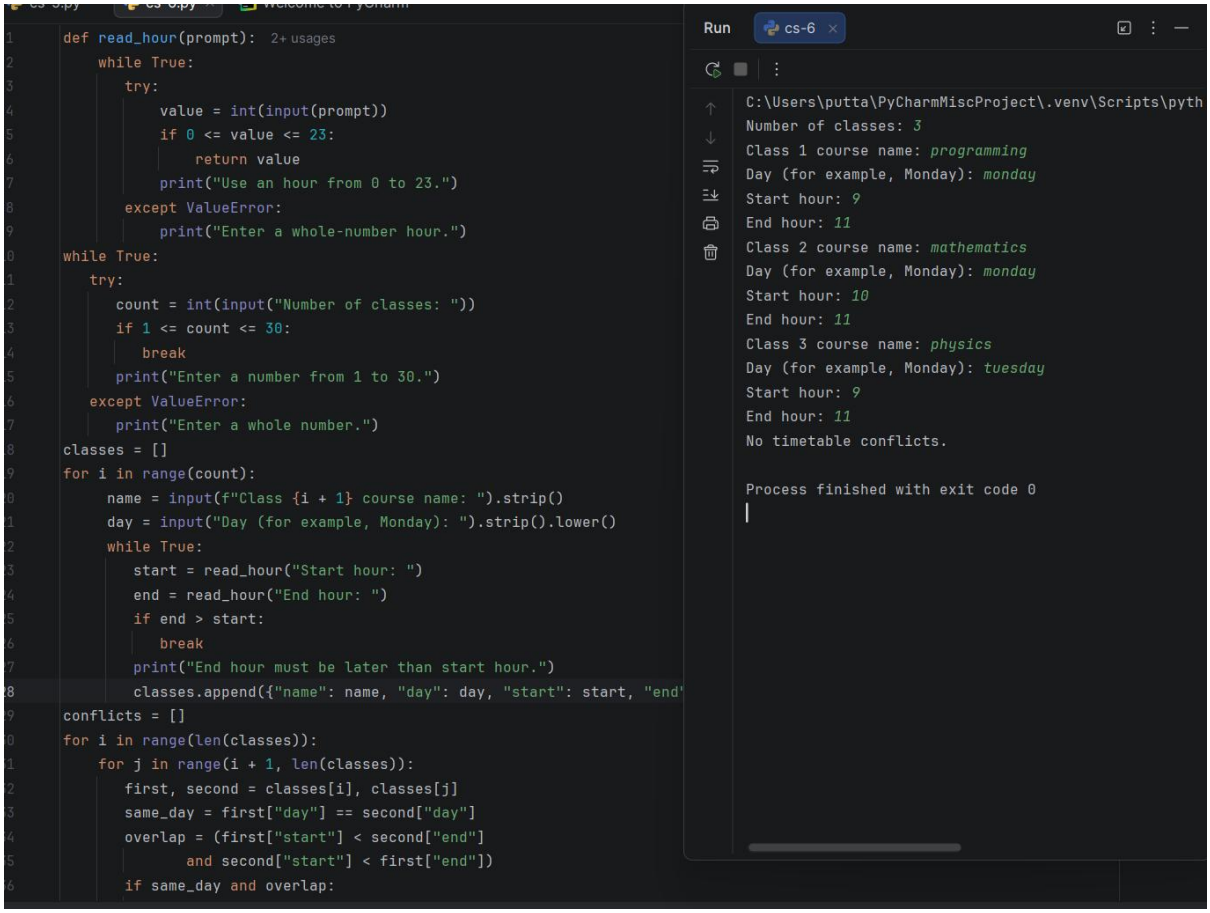
7.Test cases:

Course	Day	Start	End
Programming	Monday	9	11
Mathematics	Monday	10	12
Physics	Tuesday	9	10

8.Source code

```
def read_hour(prompt):
    while True:
       try:
           value = int(input(prompt))
           if 0 <= value <= 23:
               return value
           print("Use an hour from 0 to 23.")
       except ValueError:
           print("Enter a whole-number hour.")
while True:
   try:
      count = int(input("Number of classes: "))
      if 1 <= count <= 30:
         break
      print("Enter a number from 1 to 30.")
   except ValueError:
      print("Enter a whole number.")
classes = []
for i in range(count):
     name = input(f"Class {i + 1} course name: ").strip()
     day = input("Day (for example, Monday): ").strip().lower()
     while True:
        start = read_hour("Start hour: ")
        end = read_hour("End hour: ")
        if end > start:
           break
        print("End hour must be later than start hour.")
    classes.append({"name": name, "day": day, "start": start, "end": end})
conflicts = []
for i in range(len(classes)):
    for j in range(i + 1, len(classes)):
       first, second = classes[i], classes[j]
       same_day = first["day"] == second["day"]
       overlap = (first["start"] < second["end"]
              and second["start"] < first["end"])
       if same_day and overlap:
           conflicts.append((first["name"], second["name"], first["day"]))
if conflicts:
     print("Conflicts found:")
     for first, second, day in conflicts:
         print(f"{first} and {second} on {day}")
else:
     print("No timetable conflicts.")
```

9.output screenshot:



```
def read_hour(prompt):  2+ usages
    while True:
        try:
            value = int(input(prompt))
            if 0 <= value <= 23:
                return value
            print("Use an hour from 0 to 23.")
        except ValueError:
            print("Enter a whole-number hour.")
while True:
    try:
        count = int(input("Number of classes: "))
        if 1 <= count <= 30:
            break
        print("Enter a number from 1 to 30.")
    except ValueError:
        print("Enter a whole number.")
classes = []
for i in range(count):
    name = input(f"Class {i + 1} course name: ").strip()
    day = input("Day (for example, Monday): ").strip().lower()
    while True:
        start = read_hour("Start hour: ")
        end = read_hour("End hour: ")
        if end > start:
            break
        print("End hour must be later than start hour.")
        classes.append({"name": name, "day": day, "start": start, "end"
conflicts = []
for i in range(len(classes)):
    for j in range(i + 1, len(classes)):
        first, second = classes[i], classes[j]
        same_day = first["day"] == second["day"]
        overlap = (first["start"] < second["end"]
                and second["start"] < first["end"])
        if same_day and overlap:
```

```
Run   cs-6
C:\Users\putta\PyCharmMiscProject\.venv\Scripts\pyth
Number of classes: 3
Class 1 course name: programming
Day (for example, Monday): monday
Start hour: 9
End hour: 11
Class 2 course name: mathematics
Day (for example, Monday): monday
Start hour: 10
End hour: 11
Class 3 course name: physics
Day (for example, Monday): tuesday
Start hour: 9
End hour: 11
No timetable conflicts.

Process finished with exit code 0
```

10.expected output:

```
Number of classes: 3
Class 1 course name: programming
Day (for example, Monday): monday
Start hour: 9
End hour: 11
Class 2 course name: mathematics
Day (for example, Monday): monday
Start hour: 10
End hour: 11
Class 3 course name: physics
Day (for example, Monday): tuesday
Start hour: 9
End hour: 11
No timetable conflicts.
```

11.actual output:

```
Number of classes: 3
Class 1 course name: programming
Day (for example, Monday): monday
Start hour: 9
End hour: 11
Class 2 course name: mathematics
Day (for example, Monday): monday
Start hour: 10
End hour: 11
Class 3 course name: physics
Day (for example, Monday): tuesday
```

```
Start hour: 9
End hour: 11
No timetable conflicts.
```

12. Conclusion

This program checks a student timetable for classes that occur on the same day and overlap in time. By comparing each pair once and applying the interval rule, it reports schedule conflicts while allowing back-to-back classes. The case demonstrates how nested loops, lists, dictionaries, and input validation can solve a practical college scheduling problem.

Smart Canteen Order and Stock Management

1.Problem statement:

Create a Python program for a canteen with a fixed menu. For each order line, validate the item code and quantity. Accept an order only when the quantity is positive and available in stock. For an accepted order, reduce stock and add the item cost to the running bill. At the end, show the total bill and the remaining stock for every menu item.

2.Objectives:

- Use a dictionary to store each item's name, price, and stock.
- Validate order input and reject invalid or unavailable requests.
- Update the stock and bill only for accepted order lines.
- Practice loops, conditions, dictionaries, and formatted output.

3.Assumptions:

- The menu is fixed while the program is running: Sandwich (S) Rs 60, Juice (J) Rs 30, and Idli plate (I) Rs 40.
- Stock starts at 5 sandwiches, 8 juices, and 6 idli plates.
- Each order line contains one item code and a whole-number quantity. A customer may order the same item on separate lines if stock remains.
- No tax, discount, payment, or persistent storage is included. All currency values are shown as whole rupees.

4.Algorithm:

1. Create the menu dictionary with item names, prices, and stock quantities.
2. Display the menu and read a non-negative number of order lines.
3. For each order line, read and normalize the item code. Reject unknown codes.
4. Read the quantity. Reject non-numeric, non-positive, or above-stock quantities.
5. For an accepted order, calculate cost, reduce stock, add cost to the bill, and report the accepted line.
6. After all lines, display the bill total and each item's remaining stock.

5.Pseudocode:

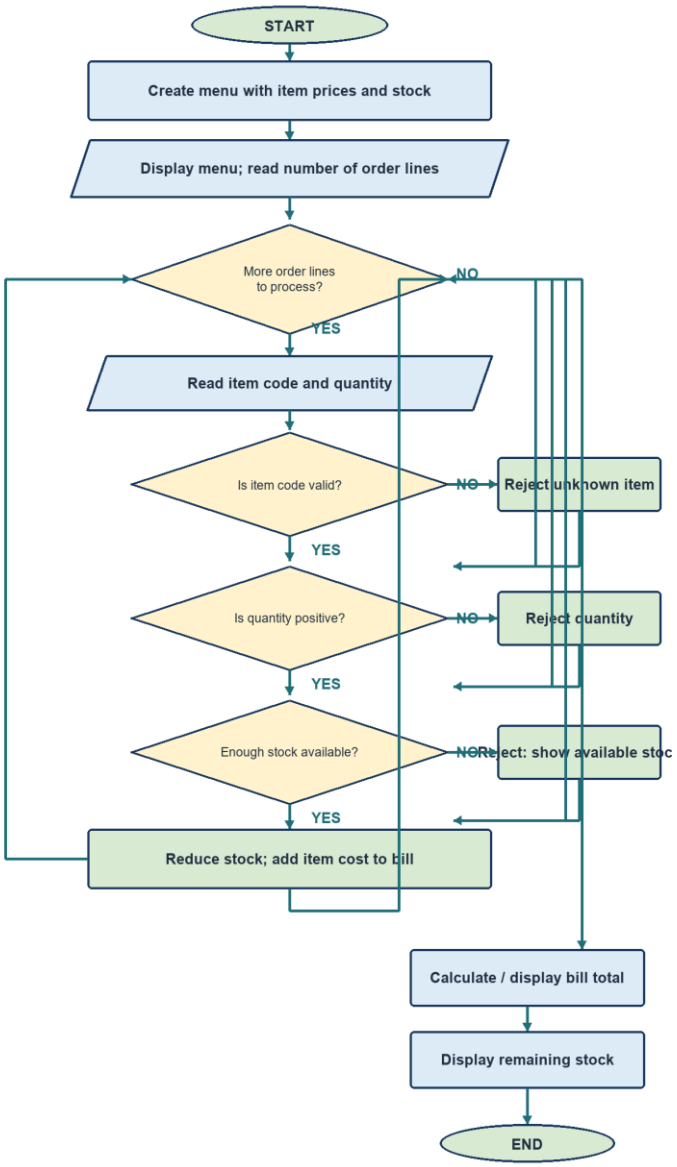
```
BEGIN
 menu = {S: Sandwich, Rs 60, stock 5; J: Juice, Rs 30, stock 8;
 I: Idli plate, Rs 40, stock 6}
 DISPLAY menu
 READ number_of_order_lines (must be zero or greater)
 total = 0


 FOR each order line
 READ item_code; convert to uppercase
 IF item_code is not in menu
```

```
 DISPLAY rejection; continue to next line
 READ quantity
 IF quantity is not a whole number or quantity <= 0
 DISPLAY rejection; continue to next line
 IF quantity > available stock
 DISPLAY available stock; continue to next line
 cost = quantity * item price
 available stock = available stock - quantity
 total = total + cost
 DISPLAY accepted item and cost


 DISPLAY total bill and remaining stock for all items
END
```

6.Flowchart:

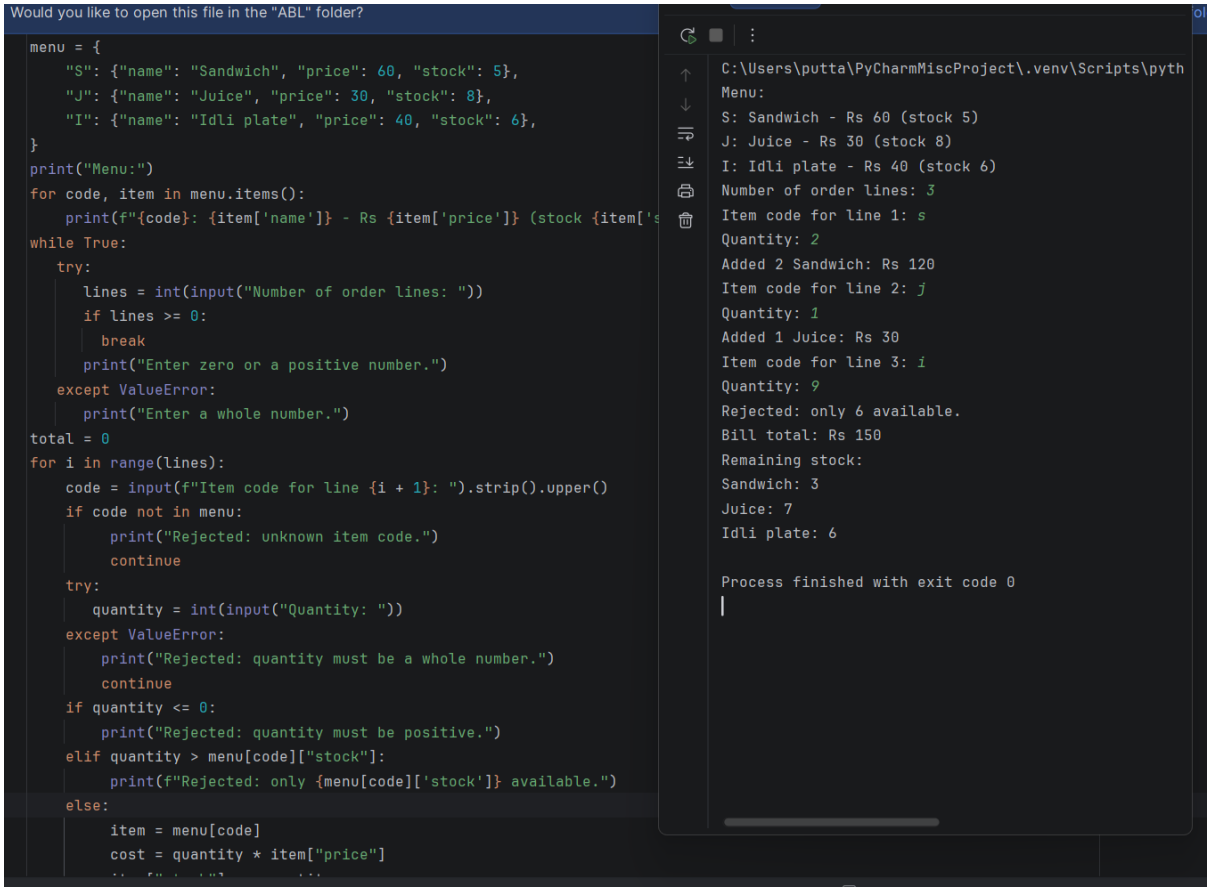


7.Source code:

```
menu = {
```

```
    "S": {"name": "Sandwich", "price": 60, "stock": 5},
    "J": {"name": "Juice", "price": 30, "stock": 8},
    "I": {"name": "Idli plate", "price": 40, "stock": 6},
}
print("Menu:")
 for code, item in menu.items():
    print(f"{code}: {item['name']} - Rs {item['price']} (stock {item['stock']})")
while True:
   try:
      lines = int(input("Number of order lines: "))
      if lines >= 0:
        break
      print("Enter zero or a positive number.")
    except ValueError:
      print("Enter a whole number.")
total = 0
for i in range(lines):
    code = input(f"Item code for line {i + 1}: ").strip().upper()
    if code not in menu:
         print("Rejected: unknown item code.")
         continue
    try:
       quantity = int(input("Quantity: "))
    except ValueError:
        print("Rejected: quantity must be a whole number.")
        continue
     if quantity <= 0:
         print("Rejected: quantity must be positive.")
     elif quantity > menu[code]["stock"]:
         print(f"Rejected: only {menu[code]['stock']} available.")
     else:
         item = menu[code]
         cost = quantity * item["price"]
         item["stock"] -= quantity
         total += cost
         print(f"Added {quantity} {item['name']}: Rs {cost}")
print(f"Bill total: Rs {total}")
print("Remaining stock:")
for code, item in menu.items():
        print(f"{item['name']}: {item['stock']}")
```

8.output screenshot:



9.expected output:

Menu:

S: Sandwich - Rs 60 (stock 5)

J: Juice - Rs 30 (stock 8)

I: Idli plate - Rs 40 (stock 6)

Number of order lines: 3

Item code for line 1: s

Quantity: 2

Added 2 Sandwich: Rs 120

Item code for line 2: j

Quantity: 1

Added 1 Juice: Rs 30

Item code for line 3: i

Quantity: 9

Rejected: only 6 available.

Bill total: Rs 150

Remaining stock:

Sandwich: 3

Juice: 7

Idli plate: 6

10. Actual output:

Menu:

S: Sandwich - Rs 60 (stock 5)

J: Juice - Rs 30 (stock 8)

I: Idli plate - Rs 40 (stock 6)

Number of order lines: 3

Item code for line 1: s

Quantity: 2

Added 2 Sandwich: Rs 120

Item code for line 2: j

Quantity: 1

Added 1 Juice: Rs 30

Item code for line 3: i

Quantity: 9

Rejected: only 6 available.

Bill total: Rs 150

Remaining stock:

Sandwich: 3

Juice: 7

Idli plate: 6

11.Test cases and expected outputs

1. Valid order

Input: S x 2, J x 1

Expected: Sandwich line Rs 120; Juice line Rs 30; bill Rs 150; remaining stock S=3, J=7, I=6.

2. Order more than stock

Input: I x 9

Expected: Rejected: only 6 available. Bill Rs 0; stock stays S=5, J=8, I=6.

3. Unknown item code

Input: X x 1

Expected: Rejected: unknown item code. Bill Rs 0; stock is unchanged.

4. Non-positive quantity

Input: J x 0

Expected: Rejected: quantity must be positive. Bill Rs 0; stock is unchanged.

5. Orders use remaining stock

Input: S x 4, then S x 2

Expected: First line accepted for Rs 240; second line rejected because only 1 sandwich remains. Bill Rs 240; S stock=1.

12.Conclusion:

The canteen program keeps the bill and inventory consistent by changing both only after an order passes the item, quantity, and stock checks. It demonstrates how dictionaries and conditional logic can model a simple real-world ordering task.

Sports Tournament Ranking System

1.Problem statement

Read each team's wins, draws, losses, goals scored, and goals conceded. Award 3 points for each win and 1 point for each draw. Rank teams by points from highest to lowest. If points are tied, compare goal difference, then goals scored, and finally team name alphabetically.

2.Objectives

- Store each team and its tournament statistics in a dictionary.
- Calculate points and goal difference from the input values.
- Sort teams using multiple tie-break rules.
- Display a clear ranked table.

3.Assumptions

- Each team has completed the same number of matches; wins, draws, losses, goals for, and goals against are non-negative whole numbers.
- A win is worth 3 points, a draw 1 point, and a loss 0 points.
- Tie-break order is points descending, goal difference descending, goals for descending, then team name alphabetically without case sensitivity.
- The program ranks the supplied records; it does not calculate standings from individual match results.

4.Algorithm

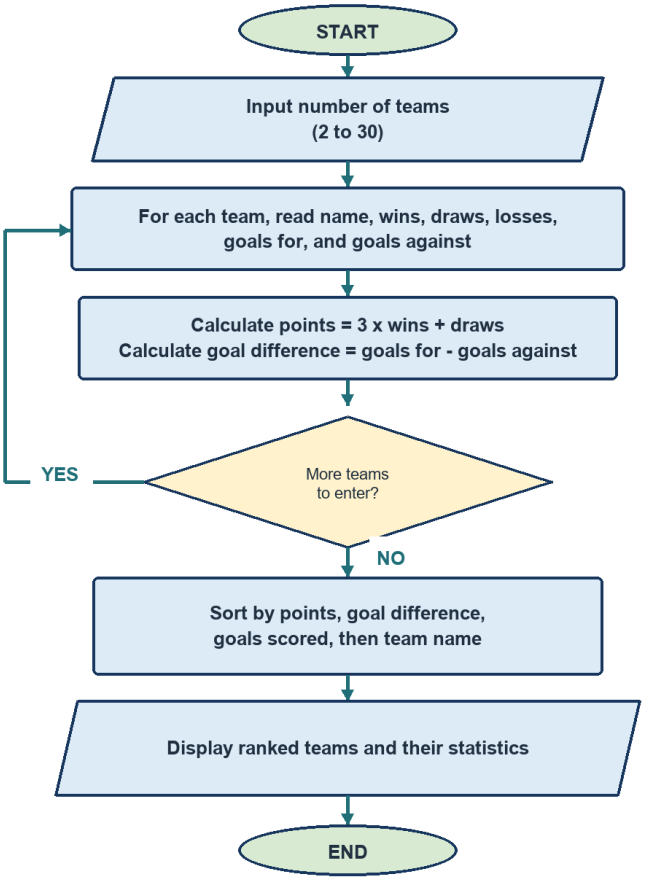
1. Read a valid number of teams.
2. For each team, read its name and statistics.
3. Calculate points as 3 times wins plus draws. Calculate goal difference as goals for minus goals against.
4. Sort the team records by points, goal difference, goals for, then name using the stated order.
5. Display each team's rank, points, goal difference, and goals for.

5.Pseudocode

```
BEGIN
 READ team_count (2 through 30)
 teams = empty list
 FOR each team
 READ name, wins, draws, losses, goals_for, goals_against
 points = 3 * wins + draws
 goal_difference = goals_for - goals_against
 STORE team record
 SORT teams by:
 points descending
 goal_difference descending
 goals_for descending
 name alphabetically
 FOR each sorted team, starting rank at 1
```

```
 DISPLAY rank, name, points, goal_difference, goals_for
END
```

6.Flowchart



7.Source code

```
def read_nonnegative_int(prompt):
while True:
try:
value = int(input(prompt))
if value >= 0:
return value
print("Enter zero or a positive whole number.")
except ValueError:
```

```
print("Enter a whole number.")
while True:
try:
team_count = int(input("Number of teams: "))
if 2 <= team_count <= 30:
break
print("Enter a number from 2 to 30.")
except ValueError:
print("Enter a whole number.")
teams = []
for i in range(team_count):
name = input(f"Team {i + 1} name: ").strip()
wins = read_nonnegative_int("Wins: ")
draws = read_nonnegative_int("Draws: ")
losses = read_nonnegative_int("Losses: ")
goals_for = read_nonnegative_int("Goals for: ")
goals_against = read_nonnegative_int("Goals against: ")
points = wins * 3 + draws
goal_difference = goals_for - goals_against
teams.append({"name": name, "points": points,
"goal_difference": goal_difference,
"goals_for": goals_for, "wins": wins})
teams.sort(key=lambda team: (-team["points"],
-team["goal_difference"],
-team["goals_for"],
team["name"].lower()))
print("Final tournament ranking:")
for rank, team in enumerate(teams, start=1):
print(f"{rank}. {team['name']} - {team['points']} points, "
f"GD {team['goal_difference']}, GF {team['goals_for']}")
```

8.Test cases and expected outputs

1. Points decide ranking

Input: Team A: 3 wins, 0 draws; Team B: 2 wins, 1 draw.

Expected: A has 9 points and ranks above B with 7 points.

2. Goal difference tie-break

Input: Team A: 2 wins, 1 draw, GF 6, GA 2; Team B: same points, GF 5, GA 3.

Expected: Both have 7 points; A has GD 4 and ranks above B with GD 2.

3. Goals-for tie-break

Input: Team A and B both have 7 points and GD 2; A GF 5, B GF 6.

Expected: B ranks above A because 6 goals for is greater than 5.

4. Alphabetical final tie-break

Input: Lions and Tigers have identical points, goal difference, and goals for.

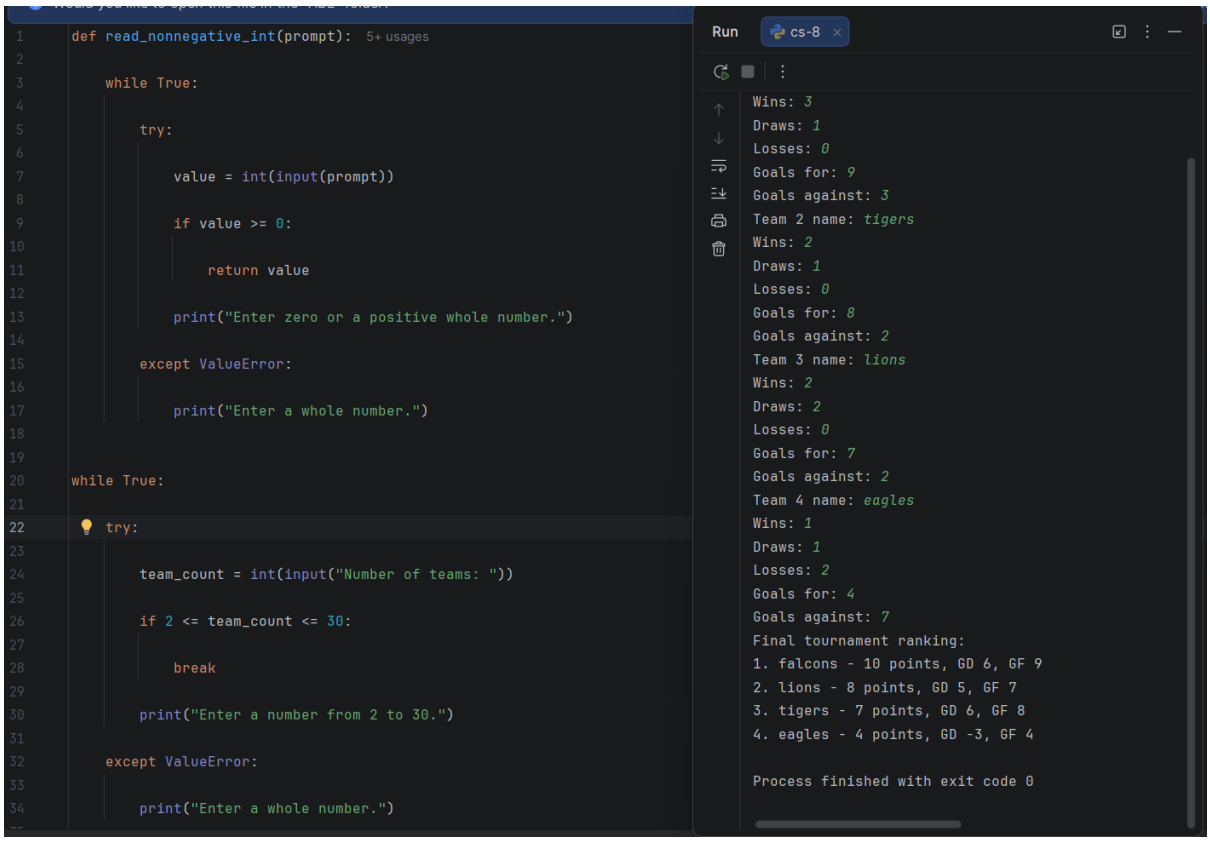
Expected: Lions ranks before Tigers.

5. Negative goal difference

Input: Eagles: 1 win, 0 draws, GF 2, GA 5.

Expected: Eagles has 3 points and GD -3; the negative difference is shown correctly.

9.output screenshot:



10.expected output

Number of teams: 4

Team 1 name: falcons

Wins: 3

Draws: 1

Losses: 0

Goals for: 9

Goals against: 3

Team 2 name: tigers

Wins: 2

Draws: 1

Losses: 0

Goals for: 8

Goals against: 2

Team 3 name: lions

Wins: 2

Draws: 2

Losses: 0

Goals for: 7

Goals against: 2

Team 4 name: eagles

Wins: 1 Draws: 1

Losses: 2

Goals for: 4

Goals against: 7

Final tournament ranking:

1. falcons - 10 points, GD 6, GF 9

2. lions - 8 points, GD 5, GF 7

3. tigers - 7 points, GD 6, GF 8

4. eagles - 4 points, GD -3, GF 4

11.actual output

Number of teams: 4

Team 1 name: falcons

Wins: 3

Draws: 1

Losses: 0

Goals for: 9

Goals against: 3

Team 2 name: tigers

Wins: 2

Draws: 1

Losses: 0

Goals for: 8

Goals against: 2

Team 3 name: lions

Wins: 2

Draws: 2

Losses: 0

Goals for: 7

Goals against: 2

Team 4 name: eagles

Wins: 1

Draws: 1

Losses: 2

Goals for: 4

Goals against: 7

Final tournament ranking:

1. falcons - 10 points, GD 6, GF 9

2. lions - 8 points, GD 5, GF 7

3. tigers - 7 points, GD 6, GF 8

4. eagles - 4 points, GD -3, GF 4

12.Conclusion

The ranking program turns team records into a consistent tournament table. Calculating points and goal difference, then sorting with explicit tie-break rules, makes the result easy to explain and reproduce.

University Scholarship Allocation

1.Problem statement

Read the number of scholarship places and each applicant's name, entrance score, and annual family income. An applicant qualifies with a score of at least 80 and annual family income no greater than Rs 500000. Rank qualified applicants by higher score, then lower family income, then name alphabetically. Award scholarships to the top applicants up to the available number of places.

2.Objectives

- Apply two conditions together to decide eligibility.
- Store eligible applicant records in a list of dictionaries.
- Sort records with multiple tie-break rules.
- Allocate a limited number of scholarships and report eligible applicants who were not selected.

3.Assumptions

- The score is an integer from 0 to 100; family income is a non-negative whole number in rupees.
- Eligibility requires both score >= 80 and income <= Rs 500000.
- The ranking order is score descending, income ascending, then name alphabetically without regard to case.
- Every scholarship place has the same value; the program selects recipients but does not calculate award amounts.

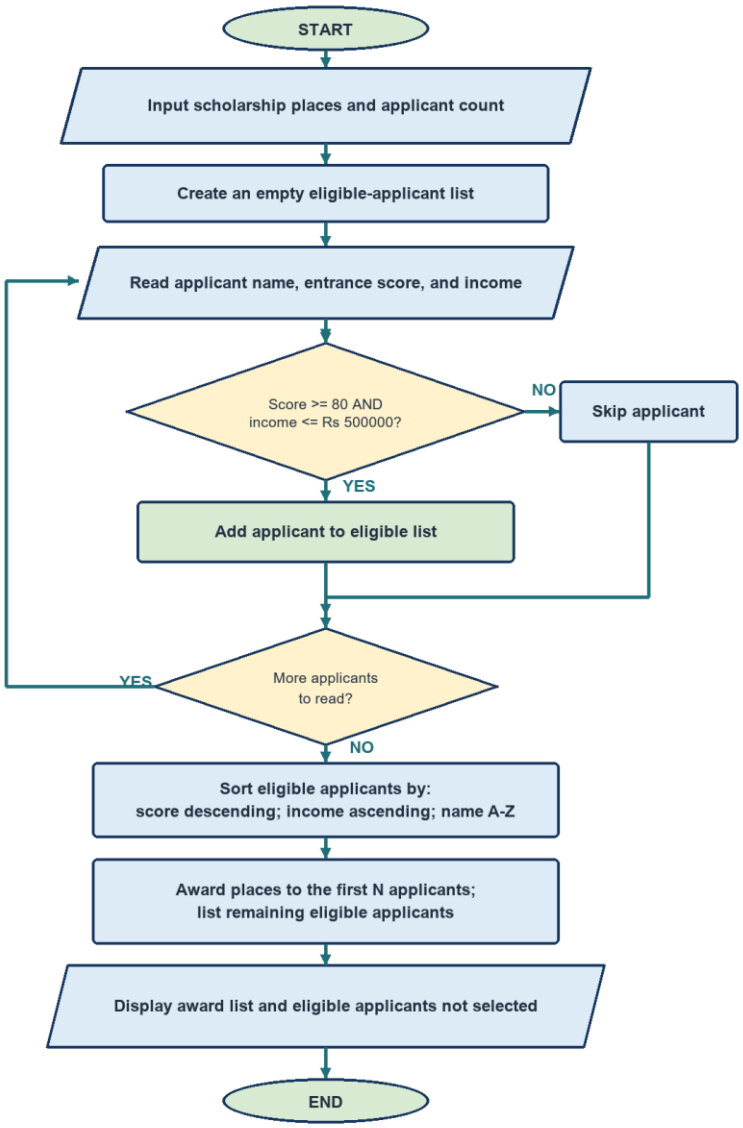
4.Algorithm

1. Read and validate the number of scholarship places and applicants.
2. For every applicant, read the name, entrance score, and annual family income.
3. If score is at least 80 and income is at most Rs 500000, add the applicant to the eligible list.
4. Sort eligible applicants by score from highest to lowest, income from lowest to highest, and name alphabetically.
5. Select the first N eligible applicants for the available N places.
6. Display awarded applicants and eligible applicants who were not selected.

5.Pseudocode

```
BEGIN
 READ number_of_places (0 through 100)
 READ number_of_applicants (1 through 100)
 eligible = empty list
 FOR each applicant
 READ name, score, annual_income
 IF score >= 80 AND annual_income <= 500000
 ADD applicant to eligible
 SORT eligible by score descending,
 income ascending, then name alphabetically
 awarded = first number_of_places applicants
 not_selected = remaining eligible applicants
 DISPLAY awarded list and not_selected list
END
```

6.Flowchart



7.Source code

```
def read_int(prompt, low, high):
while True:
try:
value = int(input(prompt))
if low <= value <= high:
return value
print(f"Enter a value from {low} to {high}.")
except ValueError:
print("Enter a whole number.")
places = read_int("Scholarship places: ", 0, 100)
count = read_int("Number of applicants: ", 1, 100)
eligible = []
for i in range(count):
name = input(f"Applicant {i + 1} name: ").strip()
score = read_int("Entrance score (0-100): ", 0, 100)
income = read_int("Annual family income in rupees: ", 0, 100000000)
if score >= 80 and income <= 500000:
```

```
eligible.append({"name": name, "score": score, "income": income})
eligible.sort(key=lambda student: (-student["score"],
student["income"],
student["name"].lower()))
awarded = eligible[:places]
not_selected = eligible[places:]
print("Scholarship awards:")
if awarded:
for rank, student in enumerate(awarded, start=1):
print(f"{rank}. {student['name']} - score {student['score']}, "
f"income Rs {student['income']}")
else:
print("No scholarships awarded.")
print("Eligible but not selected:")
if not_selected:
for student in not_selected:
print(f"{student['name']} - score {student['score']}, "
f"income Rs {student['income']}")
else:
print("None")
```

8.Test cases and expected outputs

1. Score below requirement

Input: Score 79, income Rs 200000.

Expected: Not eligible because the score is below 80.

2. Income above limit

Input: Score 90, income Rs 500001.

Expected: Not eligible because income exceeds Rs 500000.

3. Score and income tie-breaks

Input: Asha and Bala both score 91; Bala income Rs 350000, Asha Rs 400000.

Expected: Both qualify; Bala ranks ahead because family income is lower.

4. More eligible applicants than places

Input: Two places; three qualifying applicants.

Expected: Top two by ranking receive scholarships; third appears under eligible but not selected.

5. No eligible applicants

Input: All applicants score below 80 or have income above Rs 500000.

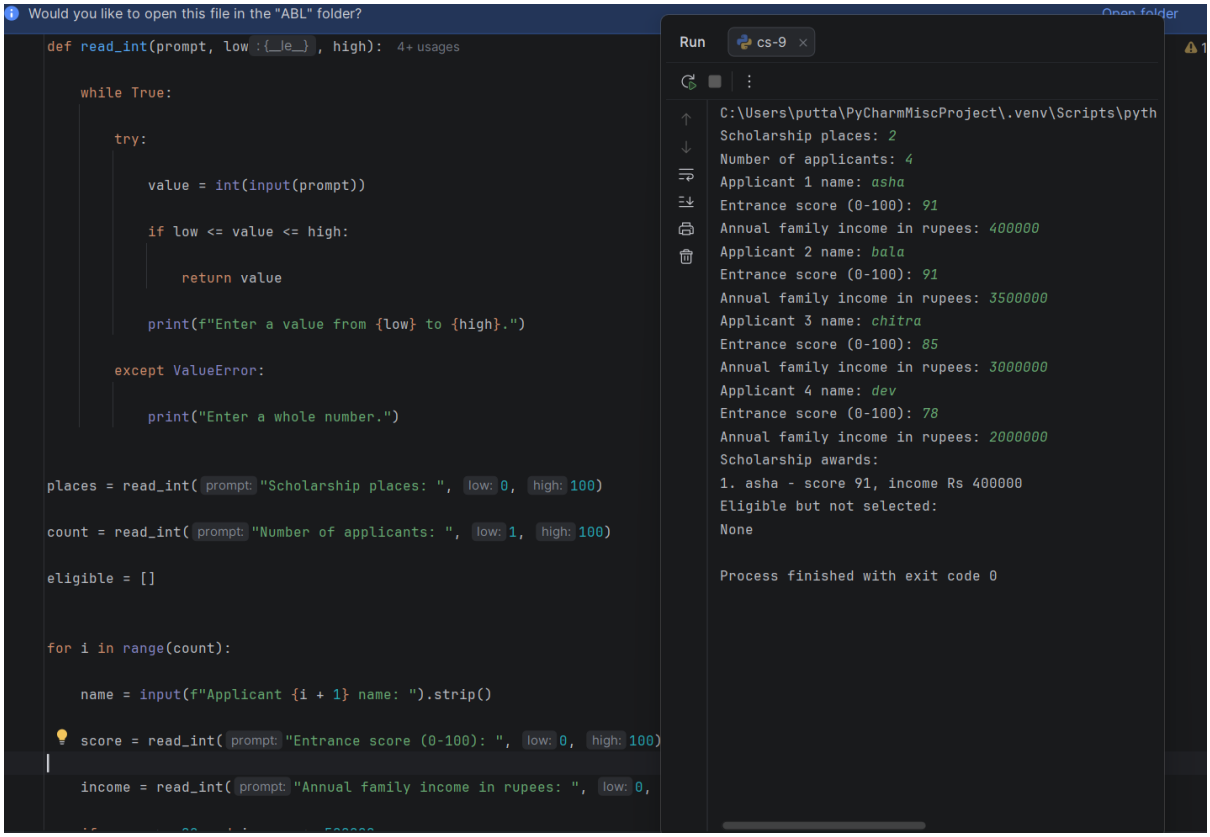
Expected: Scholarship awards: No scholarships awarded. Eligible but not selected: None.

6. No scholarship places

Input: Places set to 0; at least one applicant qualifies.

Expected: No scholarships awarded; every qualified applicant is listed as eligible but not selected.

9.output screenshot



10.expected output

Scholarship places: 2

Number of applicants: 4

Applicant 1 name: asha

Entrance score (0-100): 91

Annual family income in rupees: 400000

Applicant 2 name: bala

Entrance score (0-100): 91

Annual family income in rupees: 3500000

Applicant 3 name: chitra

Entrance score (0-100): 85

Annual family income in rupees: 3000000

Applicant 4 name: dev

Entrance score (0-100): 78

Annual family income in rupees: 2000000

Scholarship awards:

1. asha - score 91, income Rs 400000

Eligible but not selected:

None

11.actual output

Scholarship places: 2

Number of applicants: 4

Applicant 1 name: asha

Entrance score (0-100): 91

Annual family income in rupees: 400000

Applicant 2 name: bala

Entrance score (0-100): 91

Annual family income in rupees: 3500000

Applicant 3 name: chitra

Entrance score (0-100): 85

Annual family income in rupees: 3000000

Applicant 4 name: dev

Entrance score (0-100): 78

Annual family income in rupees: 2000000

Scholarship awards:

1. asha - score 91, income Rs 400000

Eligible but not selected:

None

12.Conclusion

The scholarship allocator uses clear eligibility checks and a documented ranking order to make limited awards consistently. Separating qualification, sorting, and allocation makes each decision easy to follow and verify.

University Hostel Room Allocation

1.Problem statement

Read the available number of hostel rooms and each applicant's name, distance from campus, and GPA. An applicant qualifies if their home is at least 50 km from campus and their GPA is at least 5.0 out of 10. Rank eligible applicants by greater distance first, then higher GPA, then name alphabetically. Assign rooms to the top applicants up to capacity.

2.Objectives

- Apply multiple conditions to check hostel eligibility.
- Store eligible applicants as structured records.
- Rank applicants using distance, GPA, and a final name tie-break.
- Allocate a limited resource and report both successful and unsuccessful eligible applicants.

3.Assumptions

- The eligibility thresholds are practice rules for this case: distance >= 50 km and GPA >= 5.0.
- Distance is a whole number of kilometres; GPA is a number from 0 to 10.
- Room priority is distance descending, GPA descending, then applicant name alphabetically without regard to case.
- Every room is equivalent. This program assigns room numbers in ranking order and does not handle room type, accessibility, or shared-room preferences.

4.Algorithm

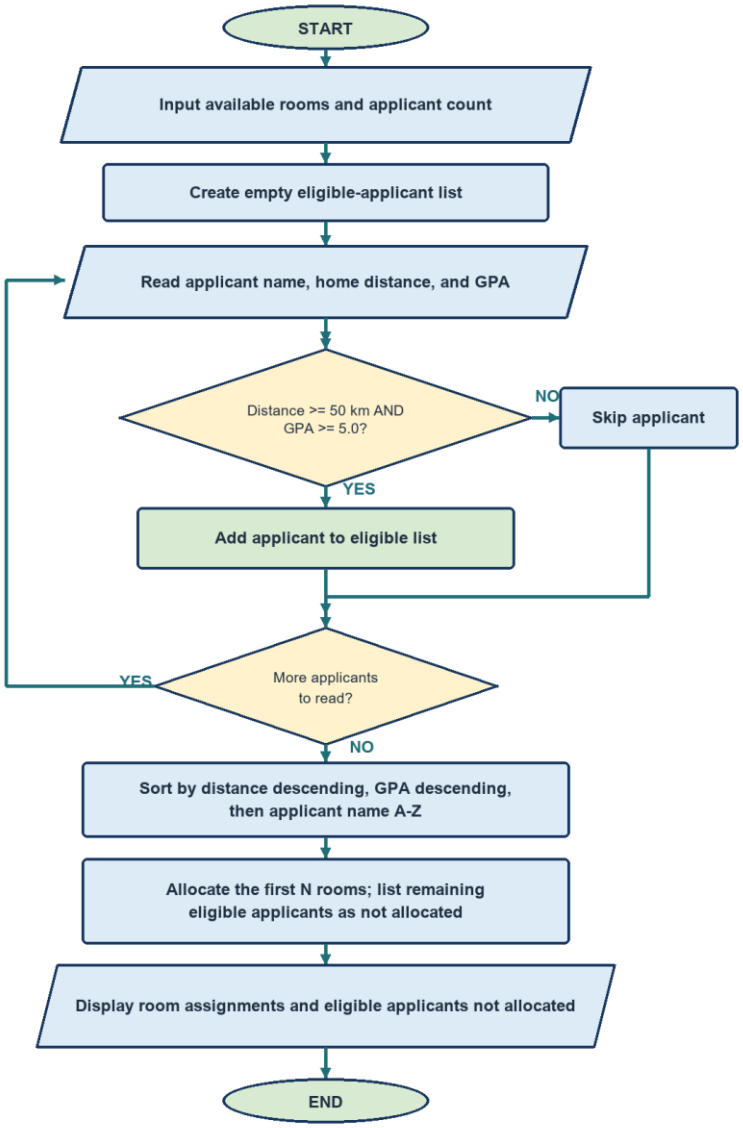
1. Read and validate room capacity and applicant count.
2. For each applicant, read their name, home distance, and GPA.
3. Add the applicant to the eligible list only if distance is at least 50 km and GPA is at least 5.0.
4. Sort eligible applicants by distance from greatest to least, GPA from highest to lowest, and name alphabetically.
5. Allocate the first N applicants to the N available rooms.
6. Display room assignments and the eligible applicants who were not allocated a room.

5.Pseudocode

```
BEGIN
 READ available_rooms (0 or more)
 READ applicant_count (1 through 100)
 eligible = empty list
 FOR each applicant
 READ name, distance_km, GPA
 IF distance_km >= 50 AND GPA >= 5.0
 ADD applicant to eligible
 SORT eligible by distance descending,
 GPA descending, then name alphabetically
 allocated = first available_rooms applicants
 not_allocated = remaining eligible applicants
```

```
 DISPLAY room assignments and not_allocated list
END
```

6.Flowchart



7.Source code

```
def read_int(prompt, low, high):
while True:
try:
value = int(input(prompt))
if low <= value <= high:
return value
print(f"Enter a whole number from {low} to {high}.")
except ValueError:
print("Enter a whole number.")
def read_gpa(prompt):
while True:
try:
value = float(input(prompt))
```

```
if 0 <= value <= 10:
return value
print("Enter a GPA from 0 to 10.")
except ValueError:
print("Enter a number.")
rooms = read_int("Available hostel rooms: ", 0, 100)
count = read_int("Number of applicants: ", 1, 100)
eligible = []
for i in range(count):
name = input(f"Applicant {i + 1} name: ").strip()
distance = read_int("Home distance from campus in km: ", 0, 100000)
gpa = read_gpa("GPA (0-10): ")
if distance >= 50 and gpa >= 5.0:
eligible.append({"name": name, "distance": distance, "gpa": gpa})
eligible.sort(key=lambda student: (-student["distance"],
-student["gpa"],
student["name"].lower()))
allocated = eligible[:rooms]
not_allocated = eligible[rooms:]
print("Hostel room allocation:")
if allocated:
for room_number, student in enumerate(allocated, start=1):
print(f"Room {room_number}: {student['name']} - "
f"{student['distance']} km, GPA {student['gpa']:.1f}")
else:
print("No rooms allocated.")
print("Eligible but not allocated:")
if not_allocated:
for student in not_allocated:
print(f"{student['name']} - {student['distance']} km, "
f"GPA {student['gpa']:.1f}")
else:
print("None")
```

8.Test cases and expected outputs

1. Applicant meets both requirements

Input: Distance 75 km, GPA 7.0.

Expected: Applicant is eligible.

2. Distance below threshold

Input: Distance 49 km, GPA 9.0.

Expected: Applicant is not eligible.

3. GPA below threshold

Input: Distance 100 km, GPA 4.9.

Expected: Applicant is not eligible.

4. Distance tie-break

Input: Two eligible applicants have the same distance; GPAs 8.0 and 9.0.

Expected: Higher-GPA applicant receives the higher room priority.

5. Capacity limit

Input: Two rooms and three eligible applicants.

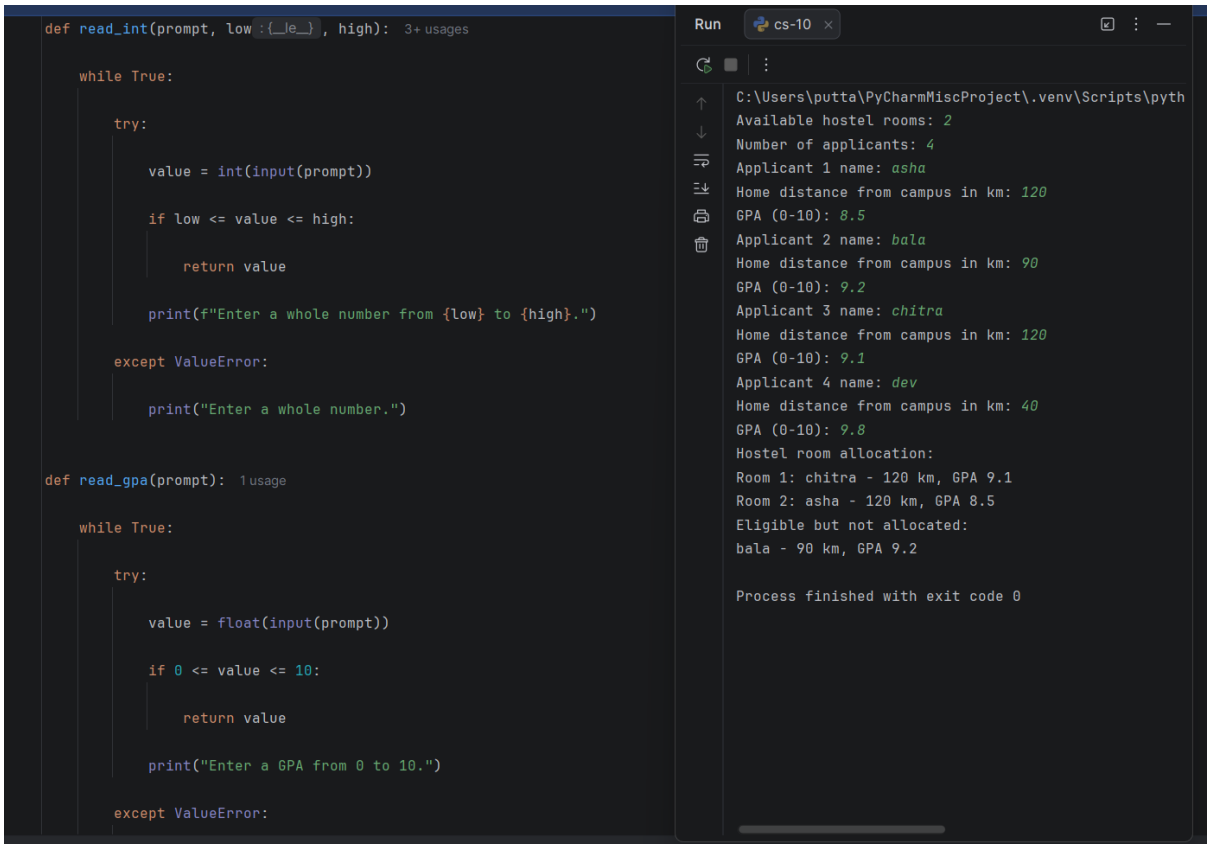
Expected: Top two by ranking receive rooms; the third is listed as eligible but not allocated.

6. No rooms available

Input: Zero rooms and at least one eligible applicant.

Expected: No rooms are allocated; eligible applicants appear in the not-allocated list.

9.outputscreenshot



```
def read_int(prompt, low:{__le__}, high):  3+ usages
    while True:
        try:
            value = int(input(prompt))
            if low <= value <= high:
                return value
            print(f"Enter a whole number from {low} to {high}.")
        except ValueError:
            print("Enter a whole number.")

def read_gpa(prompt):  1 usage
    while True:
        try:
            value = float(input(prompt))
            if 0 <= value <= 10:
                return value
            print("Enter a GPA from 0 to 10.")
        except ValueError:
```

```
Run  cs-10
C:\Users\putta\PyCharmMiscProject\.venv\Scripts\pyth
Available hostel rooms: 2
Number of applicants: 4
Applicant 1 name: asha
Home distance from campus in km: 120
GPA (0-10): 8.5
Applicant 2 name: bala
Home distance from campus in km: 90
GPA (0-10): 9.2
Applicant 3 name: chitra
Home distance from campus in km: 120
GPA (0-10): 9.1
Applicant 4 name: dev
Home distance from campus in km: 40
GPA (0-10): 9.8
Hostel room allocation:
Room 1: chitra - 120 km, GPA 9.1
Room 2: asha - 120 km, GPA 8.5
Eligible but not allocated:
bala - 90 km, GPA 9.2

Process finished with exit code 0
```

10.expected output

Available hostel rooms: 2

Number of applicants: 4

Applicant 1 name: asha

Home distance from campus in km: 120

GPA (0-10): 8.5

Applicant 2 name: bala

Home distance from campus in km: 90

GPA (0-10): 9.2

Applicant 3 name: chitra

Home distance from campus in km: 120

GPA (0-10): 9.1

Applicant 4 name: dev

Home distance from campus in km: 40

GPA (0-10): 9.8

Hostel room allocation:

Room 1: chitra - 120 km, GPA 9.1

Room 2: asha - 120 km, GPA 8.5

Eligible but not allocated:

bala - 90 km, GPA 9.2

11.actual output

Available hostel rooms: 2

Number of applicants: 4

Applicant 1 name: asha

Home distance from campus in km: 120

GPA (0-10): 8.5

Applicant 2 name: bala

Home distance from campus in km: 90

GPA (0-10): 9.2

Applicant 3 name: chitra

Home distance from campus in km: 120

GPA (0-10): 9.1

Applicant 4 name: dev

Home distance from campus in km: 40

GPA (0-10): 9.8

Hostel room allocation:

Room 1: chitra - 120 km, GPA 9.1

Room 2: asha - 120 km, GPA 8.5

Eligible but not allocated:

bala - 90 km, GPA 9.2

12.Conclusion

The hostel allocator checks eligibility before ranking applicants and assigns rooms only up to the available capacity. Explicit distance, GPA, and name tie-breaks make the allocation consistent and easy to explain.